



UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

ESCUELA DE POSGRADO

Proyecto Parcial: FinZen
Aplicación de Finanzas Personales con Consultas Inteligentes (IA)

AUTOR(ES)

Sebastian Rodrigo García Villacorta
Dante Guillermo Barreto Daza
Clinton Michael Chulluncuy Reynoso
Ronald Ramiro Hilario Orihuela

DOCENTE

Mejia Fernandez, Oscar Rodolfo

CURSO

Cloud Computing (MCD8007)

Lima – Perú

2026

Índice

1. Introducción y resumen ejecutivo	3
Parte A: Análisis (QUÉ)	4
2. Empresa y producto propuesto	4
3. Benchmark — Comparación con la competencia	4
4. Alcance funcional de FinZen (100 % digital)	5
5. Propuesta de valor	6
6. Historias de Usuario y criterios de aceptación (Gherkin/BDD)	7
6.1. HU1 (MVP): Registro y Login	7
6.2. HU2 (MVP): Registrar Ingreso / Sueldo	7
6.3. HU3 (MVP): Registrar Gasto por categoría	8
6.4. HU4 (MVP): Listado y detalle de Ingresos	8
6.5. HU5 (MVP): Listado y detalle de Gastos	8
6.6. HU6 (MVP): Editar Perfil de Usuario	9
6.7. HU7 (MVP): Consulta Inteligente de finanzas (IA / embeddings)	9
6.8. HU8 (MVP): Dashboard con resumen financiero	10
Parte B: Diseño del MVP (CÓMO)	11
7. Diseño de Frontend (Prototipo) — Next.js	11
8. Diseño de Backend — FastAPI en contenedores Docker	11
8.1. Estructura de la base de datos (SQL — SQLite)	12
8.2. Selección del motor de base de datos: SQLite frente a PostgreSQL	16
9. Empaquetado con Docker	17

10.Diagrama de Arquitectura de Solución en AWS	18
10.1. Red en AWS (VPC, Internet Gateway, subred y Security Group)	19
11.Selección de región AWS	19
12.Costos mensuales estimados en AWS	20
12.1. Costos de OpenAI (aparte de AWS)	20
 Parte C: Implementación del MVP	 22
13.Catálogo de APIs	22
14.Implementación del Backend	23
14.1. Configuración de la clave de OpenAI	23
15.Evidencia de uso con Postman	25
16.Frontend conectado (BFF + JWT)	34
17.Despliegue en AWS	39
17.1. Recursos de la instancia y uso de swap	43
18.Repositorios públicos de GitHub	43
19.Conclusiones	44
19.1. Trabajos Futuros	44
Referencias	44
Anexos	45

1. Introducción y resumen ejecutivo

FinZen constituye una aplicación web de finanzas personales, de naturaleza 100 % digital, con un asistente de IA que responde preguntas en lenguaje natural sobre las propias transacciones del usuario (por ejemplo: “¿en qué categoría gasto más?”). El sistema permite registrar ingresos y gastos por categoría, visualizar un dashboard con el balance y un gráfico de gastos, y consultar a un asistente basado en **RAG** (Retrieval-Augmented Generation) sobre embeddings.

El backend está compuesto por **4 microservicios en Python (FastAPI)**, cada uno con su **propia base de datos SQLite**, empaquetados en **contenedores Docker** y publicados detrás de **Nginx** como reverse proxy. El frontend es una aplicación **Next.js** que consume las APIs a través de un patrón **BFF** con autenticación **JWT**. Toda la solución se despliega sobre **una instancia EC2 en AWS** (región us-east-1) con **IP elástica**.

Alcance y objetivos

- Diseñar y documentar la propuesta (Parte A), el diseño del MVP (Parte B) y su implementación en contenedores (Parte C).
- Aplicar buenas prácticas de microservicios, contenedores y seguridad (JWT, contraseñas con hash, CORS restringido, usuario no root en Docker).
- Implementar un asistente RAG con embeddings, búsqueda híbrida y guardrails anti-alucinación.
- Desplegar en AWS EC2 y evidenciar el uso de las APIs con Postman.

Repositorios

- Backend: <https://github.com/UTEC-AII/finzen-app>
- Frontend: <https://github.com/UTEC-AII/finzen-webui>
- Documentación: <https://github.com/UTEC-AII/finzen-docs>

Parte A: Análisis (QUÉ)

2. Empresa y producto propuesto

- **Empresa:** FinZen Digital (empresa ficticia).
- **Producto/servicio 100 % digital:** FinZen — Finanzas personales con asistente de IA.

FinZen permite registrar ingresos, sueldo y gastos diarios y, posteriormente, formular preguntas en lenguaje natural sobre la propia información financiera, mediante un motor de búsqueda semántica basado en *embeddings*.

3. Benchmark — Comparación con la competencia

Producto	Tipos de clientes	Funcionalidades	Propuesta de valor
PocketGuard	Personas con presupuesto ajustado que buscan evitar sobregiros	Cálculo del saldo disponible tras gastos fijos; límites por categoría; vínculo bancario; alertas	Simplicidad: indica cuánto se puede gastar hoy sin sobregirarse
Copilot Money	Usuarios de Apple (iOS/macOS) con ingresos medios y altos	Categorización automática con ML; detección de suscripciones; presupuestos adaptativos; asistente conversacional	IA que aprende del usuario y reduce la categorización manual
Monarch Money	Individuos y parejas que consolidan sus finanzas	Vínculo a más de 10 000 instituciones; patrimonio neto; metas compartidas; asistente de IA	Gestión integral y colaborativa en una sola aplicación
YNAB	Personas que adoptan un método de presupuesto por sobres	Presupuesto base cero; asignación por categorías; metas de ahorro; reportes; sincronización bancaria	Disciplina financiera: asignar un destino a cada unidad monetaria
FinZen (propuesto)	Personas y freelancers en Latinoamérica con movimientos en varias monedas	Registro manual multi-moneda; panel de control; asistente RAG en lenguaje natural	Análisis en lenguaje natural sin vincular cuentas bancarias ni compartir credenciales

Cuadro 1: Cuadro comparativo del benchmark (competidores y FinZen).

Lectura del benchmark. El mercado converge en dos ejes: la **simplicidad** (PocketGuard) y la **automatización con IA** (Copilot Money, Monarch Money y YNAB). Sin embargo, todos comparten tres limitaciones que dejan un espacio sin cubrir:

- **Dependen de vincular cuentas bancarias:** sin esa conexión su IA no funciona, y muchos usuarios no quieren compartir sus credenciales.
- **Explican poco:** ofrecen cifras (cuánto se puede gastar o cuánto se gastó), pero no responden preguntas abiertas en lenguaje natural.
- **Asumen una sola moneda y un solo país:** no están pensadas para usuarios que mueven dinero en varias monedas, algo frecuente en Latinoamérica.

Oportunidad para FinZen. Esa combinación —lenguaje natural, sin vincular cuentas y multi-moneda— es precisamente el espacio sin cubrir. El benchmark no solo valida la idea: muestra *dónde* está el diferenciador.

4. Alcance funcional de FinZen (100 % digital)

Tipos de clientes / usuarios:

- Personas asalariadas con sueldo fijo mensual.
- Freelancers / independientes con ingresos variables.
- Estudiantes o jóvenes con mesada o ingresos ocasionales.

El MVP comprende las 8 historias de usuario priorizadas (HU1–HU8), que suman 22 story points distribuidos en dos sprints cortos. Esta selección se justifica por negocio: el usuario final requiere, como ciclo completo, identificarse (HU1), registrar sus ingresos (HU2) y gastos (HU3), consultar el detalle de ambos (HU4, HU5), ajustar su perfil (HU6) y, sobre sus propios movimientos, contar con un panel de resumen financiero (HU8) alimentado por un asistente de consulta inteligente (HU7).

Funcionalidades (Historias de Usuario) — backlog priorizado:

#	Historia de Usuario	Prioridad	Pts	Depende de	Sprint
HU1	Registro y Login de usuario	Alta	3	—	Semana 1
HU2	Registrar Ingreso / Sueldo	Alta	3	HU1	Semana 1–2
HU3	Registrar Gasto por categoría	Alta	3	HU1	Semana 1–2
HU4	Listado y detalle de Ingresos	Media	2	HU2	Semana 2
HU5	Listado y detalle de Gastos	Media	2	HU3	Semana 2
HU6	Editar Perfil de Usuario	Baja	1	HU1	Semana 2
HU7	Consulta Inteligente (IA / embeddings)	Alta	5	HU2, HU3	Semana 3
HU8	Dashboard con resumen financiero	Alta	3	HU2, HU3, HU7	Semana 3–4

Cuadro 2: Backlog priorizado del MVP (total: 22 story points).

5. Propuesta de valor

El problema. Entender las finanzas personales sigue siendo tedioso: las aplicaciones líderes exigen vincular cuentas bancarias, adoptar un método de presupuesto o interpretar gráficos. Para quien maneja varias monedas, ninguna responde a la pregunta más simple: *«¿en qué se me va el dinero?»*.

La propuesta. FinZen convierte los movimientos del usuario en **respuestas claras y conversacionales, en español y en lenguaje natural**. El usuario registra sus ingresos y gastos (en cualquier moneda) y le pregunta al asistente; este responde con base en **sus propios datos**, citando los movimientos que sustentan la respuesta.

Diferenciadores frente a la competencia:

- **Sin fricción y sin riesgo:** no vincula cuentas bancarias ni solicita credenciales.
- **IA que responde, no que solo grafica:** recuperación semántica (text-embedding-3-large) y generación aumentada (RAG) con citas y *guardrails* anti-alucinación.
- **Pensada para Latinoamérica:** multi-moneda (PEN, USD, ...) y zona horaria configurable.
- **Costo casi nulo:** el consumo de IA ronda $\approx \$0.04$ al mes para el volumen del proyecto.

En una frase: FinZen es *“pregúntale a tus finanzas”*: la forma más rápida de entender tu dinero, sin compartir tus cuentas.

6. Historias de Usuario y criterios de aceptación (Gherkin/BDD)

6.1. HU1 (MVP): Registro y Login

Como persona sin cuenta, **quiero** crear una cuenta e iniciar sesión con correo y contraseña, **para** acceder de forma segura a mi información.

Criterios de aceptación:

- **GIVEN** el usuario no tiene cuenta, **WHEN** completa nombre, correo, contraseña, moneda y país/zona horaria, **AND** presiona “Registrarse”, **THEN** se crea su perfil y accede a su dashboard.
- **GIVEN** el correo ya existe en `users.db`, **WHEN** intenta registrarse, **THEN** se muestra “correo ya registrado” y no se crea una cuenta duplicada.
- **GIVEN** una cuenta existente, **WHEN** ingresa credenciales correctas, **THEN** accede con sesión iniciada.
- **GIVEN** contraseña incorrecta, **WHEN** presiona “Entrar”, **THEN** se muestra un error genérico (sin revelar si el correo existe).

Tareas técnicas: Frontend `/register` y `/login`; Backend `POST /users` (hash bcrypt) y `POST /auth/login` (JWT); tabla `users` en `users.db`.

6.2. HU2 (MVP): Registrar Ingreso / Sueldo

Como usuario con sesión, **quiero** registrar un ingreso con tipo, monto, moneda, fecha y descripción, **para** llevar un registro completo.

Criterios de aceptación:

- **GIVEN** sesión iniciada, **WHEN** registra el ingreso, **THEN** se guarda en `incomes.db` y se envía a vectorizar.
- **GIVEN** monto vacío o negativo, **WHEN** guarda, **THEN** se muestra error de validación y no se registra.
- **GIVEN** el ingreso guardado, **WHEN** falla el `ai-service`, **THEN** el ingreso queda igualmente registrado (no bloquea al usuario).

Tareas técnicas: ruta `/incomes/new`; POST `/incomes`; integración con POST `/vectorize`; tabla `incomes`.

6.3. HU3 (MVP): Registrar Gasto por categoría

Como usuario con sesión, **quiero** registrar un gasto por categoría, **para** conocer el destino de mis gastos.

Criterios de aceptación:

- **GIVEN** sesión iniciada, **WHEN** registra el gasto, **THEN** se guarda en `expenses.db` y se vectoriza.
- **GIVEN** sin categoría seleccionada, **WHEN** guarda, **THEN** el sistema exige seleccionar una categoría.

Tareas técnicas: ruta `/expenses/new`; POST `/expenses`; GET `/expenses/categories/list`; tabla `expenses`.

6.4. HU4 (MVP): Listado y detalle de Ingresos

Como usuario con ingresos, **quiero** ver el listado y detalle, **para** revisar de dónde proviene mi dinero.

Criterios de aceptación:

- **GIVEN** ingresos registrados, **WHEN** accede a “Mis Ingresos” y selecciona uno, **THEN** ve el listado (ordenado por fecha) y el detalle completo.
- **GIVEN** sin ingresos, **WHEN** accede, **THEN** ve un estado vacío que invita a registrar el primero.

Tareas técnicas: rutas `/incomes` y `/incomes/{id}`; GET `/incomes/{user_id}`, GET `/incomes/detail/{id}`, DELETE `/incomes/{id}`.

6.5. HU5 (MVP): Listado y detalle de Gastos

Como usuario con gastos, **quiero** ver, filtrar y detallar mis gastos, **para** entender en qué gasto mi dinero.

Criterios de aceptación:

- **GIVEN** gastos registrados, **WHEN** accede a “Mis Gastos” y filtra por categoría o fecha, **THEN** ve el listado filtrado y el detalle del gasto seleccionado.
- **GIVEN** un filtro sin coincidencias, **WHEN** aplica, **THEN** se muestra un mensaje de “sin resultados”.

Tareas técnicas: rutas `/expenses` y `/expenses/{id}` (con eliminar); `GET /expenses/{user_id}` (filtros), `GET /expenses/detail/{id}`, `DELETE /expenses/{id}`.

6.6. HU6 (MVP): Editar Perfil de Usuario

Como usuario, **quiero** editar nombre, moneda, país/zona horaria y meta de ahorro, **para** mantener actualizada mi información.

Criterios de aceptación:

- **GIVEN** sesión iniciada, **WHEN** edita y guarda, **THEN** se actualizan sus datos (también la clave de OpenAI para el asistente).
- **GIVEN** intenta modificar el correo, **WHEN** está deshabilitado, **THEN** el correo permanece sin cambios.

Tareas técnicas: ruta `/profile`; `GET /users/{id}`, `PUT /users/{id}`.

6.7. HU7 (MVP): Consulta Inteligente de finanzas (IA / embeddings)

Como usuario con movimientos, **quiero** preguntar en lenguaje natural a un asistente sobre mis finanzas, **para** entender mis hábitos sin analizar los datos manualmente.

Criterios de aceptación:

- **GIVEN** movimientos registrados, **WHEN** escribe una pregunta (ej. “¿en qué categoría gasto más?”), **THEN** el sistema vectoriza la pregunta con `text-embedding-3-large`, recupera los movimientos más relevantes (búsqueda híbrida: similitud coseno + FTS5) y genera una respuesta en lenguaje natural.
- **GIVEN** el usuario sin movimientos, **WHEN** pregunta, **THEN** el asistente indica que no hay datos suficientes.
- **GIVEN** una pregunta fuera de contexto, **WHEN** no hay transacciones relevantes, **THEN** el asistente responde que no puede ayudar, sin inventar datos (guardrail).

Tareas técnicas: ruta `/assistant`; `ai-service`: `POST /vectorize` (interno), `POST /query`, `POST /reindex`, `GET/PUT/DELETE /settings/openai-key`; tablas `vectorized_transactions` y `settings` en `vectors.db`; pre-filtrado por metadata (categoría, tipo y fechas según la zona horaria del usuario).

6.8. HU8 (MVP): Dashboard con resumen financiero

Como usuario, **quiero** ver un resumen al entrar, **para** tener una visión general de mi balance y gastos.

Criterios de aceptación:

- **GIVEN** sesión iniciada, **WHEN** accede al Dashboard, **THEN** ve su balance (ingresos – gastos) por moneda, un gráfico de gastos por categoría y accesos directos.
- **GIVEN** sin movimientos, **WHEN** accede, **THEN** el balance es 0 y se le invita a registrar su primer movimiento.

Tareas técnicas: ruta `/dashboard`; agregación de datos de `GET /incomes/{user_id}` y `GET /expenses/{user_id}`.

Parte B: Diseño del MVP (CÓMO)

7. Diseño de Frontend (Prototipo) — Next.js

El Frontend se construye en **Next.js (App Router) + TypeScript + Tailwind CSS** y corre como un **contenedor Docker** sobre la misma instancia EC2, consumiendo los microservicios. A partir de las 8 historias se definieron estas rutas:

Ruta (Next.js)	Vista	HU
/	Bienvenida / Landing	—
/login	Inicio de sesión	HU1
/register	Creación de cuenta (moneda y zona horaria)	HU1
/dashboard	Balance por moneda, gráfico, accesos directos	HU8
/incomes/new	Registrar ingreso	HU2
/incomes, /incomes/[id]	Listado y detalle de ingresos	HU4
/expenses/new	Registrar gasto	HU3
/expenses, /expenses/[id]	Listado (filtros) y detalle	HU5
/profile	Editar perfil (y clave de OpenAI)	HU6
/assistant	Chat de Consulta Inteligente	HU7

Cuadro 3: Rutas del prototipo (frontend).

El prototipo responsivo (`FinZen_mockup.html`) y el prompt usado en herramientas de IA están en `frontend-prompt/` del repositorio del frontend.

8. Diseño de Backend — FastAPI en contenedores Docker

El Backend se implementa con **FastAPI**, como **4 contenedores Docker independientes** orquestados mediante **Docker Compose**. Cada contenedor sigue el principio de responsabilidad única y mantiene su propia base de datos **SQLite** persistida en un volumen.

Microservicio	Puerto	BD	Métodos principales
user-service	8001	users.db	POST /users, POST /auth/login, GET /users/{id}, PUT /users/{id}
income-service	8002	incomes.db	POST /incomes, GET /incomes/{user_id}, GET /incomes/detail/{id}, DELETE /incomes/{id}
expense-service	8003	expenses.db	POST /expenses, GET /expenses/{user_id}, GET /expenses/detail/{id}, GET /expenses/categories/list, DELETE /expenses/{id}
ai-service	8004	vectors.db	POST /vectorize, POST /query, POST /reindex, GET/PUT/DELETE /settings/openai-key

Cuadro 4: Microservicios del backend.

Un contenedor **Nginx** (en el mismo `docker-compose.yml`) actúa como reverse proxy: expone el Frontend en `/` y enruta `/api/users`, `/api/incomes`, `/api/expenses` y `/api/ai` a los puertos 8001–8004, resolviéndolos por el **nombre del servicio** en la red interna.

En lugar de conectar el Frontend directamente a cada microservicio, se centraliza todo tras **Nginx** por tres razones: el navegador solo necesita conocer **una URL y un puerto** (80); los puertos de los microservicios (8001–8004 y 3000) quedan protegidos en la red interna de Docker y no se exponen al exterior; y el enrutado por ruta (`/api/...`) se hace contra los nombres internos de los servicios, sin IP ni puertos extra expuestos.

8.1. Estructura de la base de datos (SQL — SQLite)

Cada microservicio posee su propia base SQLite y no comparte tablas con los demás. Esta decisión responde a tres razones: (i) **autonomía del servicio** (cada uno persiste solo su propio contexto de negocio), (ii) **desacoplamiento** (modificar el esquema de gastos no afecta a usuarios ni a ingresos) y (iii) **aislamiento de fallos** (un error de datos en un servicio no corrompe a los demás). Las referencias entre tablas de distintos servicios son **lógicas** —por `user_id`— y no existen foreign keys físicas entre bases distintas.

Sobre el almacenamiento: cada base es un archivo `.db` en `/app/data` dentro del contenedor y se persiste en un **volumen nombrado** de Docker (`users_data`, `incomes_data`, `expenses_data`, `vectors_data`). Esos volúmenes residen sobre el volumen **EBS (gp3)** de la instancia, de

modo que los datos sobreviven a un reinicio o a la recreación del contenedor.

- **user-service:** `users.db` (volumen `users_data`): tabla `users` con `id`, `name`, `email`, `password_hash`, `preferred_currency`, `timezone`, `monthly_savings_goal`, `created_at`.
- **income-service:** `incomes.db` (volumen `incomes_data`): tabla `incomes` con `id`, `user_id`, `type`, `amount`, `currency`, `date`, `description`, `created_at`.
- **expense-service:** `expenses.db` (volumen `expenses_data`): tabla `expenses` con `id`, `user_id`, `category`, `amount`, `currency`, `date`, `description`, `created_at`.
- **ai-service:** `vectors.db` (volumen `vectors_data`): tablas `vectorized_transactions` (`id`, `user_id`, `record_type`, `category`, `currency`, `amount`, `date`, `text`, `embedding`, `embedding_model`, `created_at`) y `settings` (`key`, `value`).

Las columnas van en inglés y los montos se guardan como `Numeric(12,2)` (Decimal), no como `Float`, para no perder precisión financiera.

```
// FinZen - Esquema de base de datos (DBML para dbdiagram.io)
// 4 microservicios, cada uno con su propia base SQLite.
// Las referencias son "lógicas" (no hay FK física entre bases distintas).

Table users {
  id varchar [pk]
  name varchar [not null]
  email varchar [not null, unique]
  password_hash varchar [not null]
  preferred_currency varchar [default: 'PEN']
  timezone varchar [default: 'America/Lima']
  monthly_savings_goal numeric(12,2) [default: 0]
  created_at varchar [not null]
  Note: 'user-service - users.db'
}

Table incomes {
  id varchar [pk]
  user_id varchar [not null]
  type varchar [not null] // Sueldo | Freelance | Bono | Inversion | Otro
  amount numeric(12,2) [not null]
  currency varchar [not null]
  date date [not null]
  description varchar
  created_at varchar [not null]
```

```
Note: 'income-service - incomes.db'
```

```
indexes {
  (user_id, date) [name: 'idx_incomes_user_date']
  (user_id, type) [name: 'idx_incomes_user_type']
}
}
```

```
Table expenses {
  id varchar [pk]
  user_id varchar [not null]
  category varchar [not null] // Alimentacion | Transporte | Vivienda |
  ...
  amount numeric(12,2) [not null]
  currency varchar [not null]
  date date [not null]
  description varchar
  created_at varchar [not null]
Note: 'expense-service - expenses.db'
```

```
indexes {
  (user_id, date) [name: 'idx_expenses_user_date']
  (user_id, category) [name: 'idx_expenses_user_category']
}
}
```

```
Table vectorized_transactions {
  id varchar [pk]
  user_id varchar [not null]
  record_type varchar [not null] // income | expense
  category varchar
  currency varchar
  amount numeric(12,2)
  date varchar
  description varchar
  text varchar [not null] // frase vectorizada (incluye moneda)
  embedding text [not null] // vector de 3072 dims (JSON)
  embedding_model varchar // text-embedding-3-large | local-hash
  created_at varchar [not null]
Note: 'ai-service - vectors.db'
```

```
indexes {
  (user_id, record_type) [name: 'idx_vectors_user_type']
}
}
```

```
}

Table settings {
  key varchar [pk]
  value text [not null]
  Note: 'ai-service - vectors.db (clave de OpenAI, etc.)'
}

// Relaciones lógicas (FK por user_id entre microservicios)
Ref: incomes.user_id > users.id
Ref: expenses.user_id > users.id
Ref: vectorized_transactions.user_id > users.id
```

Código 1: Esquema de las bases de datos (DBML — dbdiagram.io).

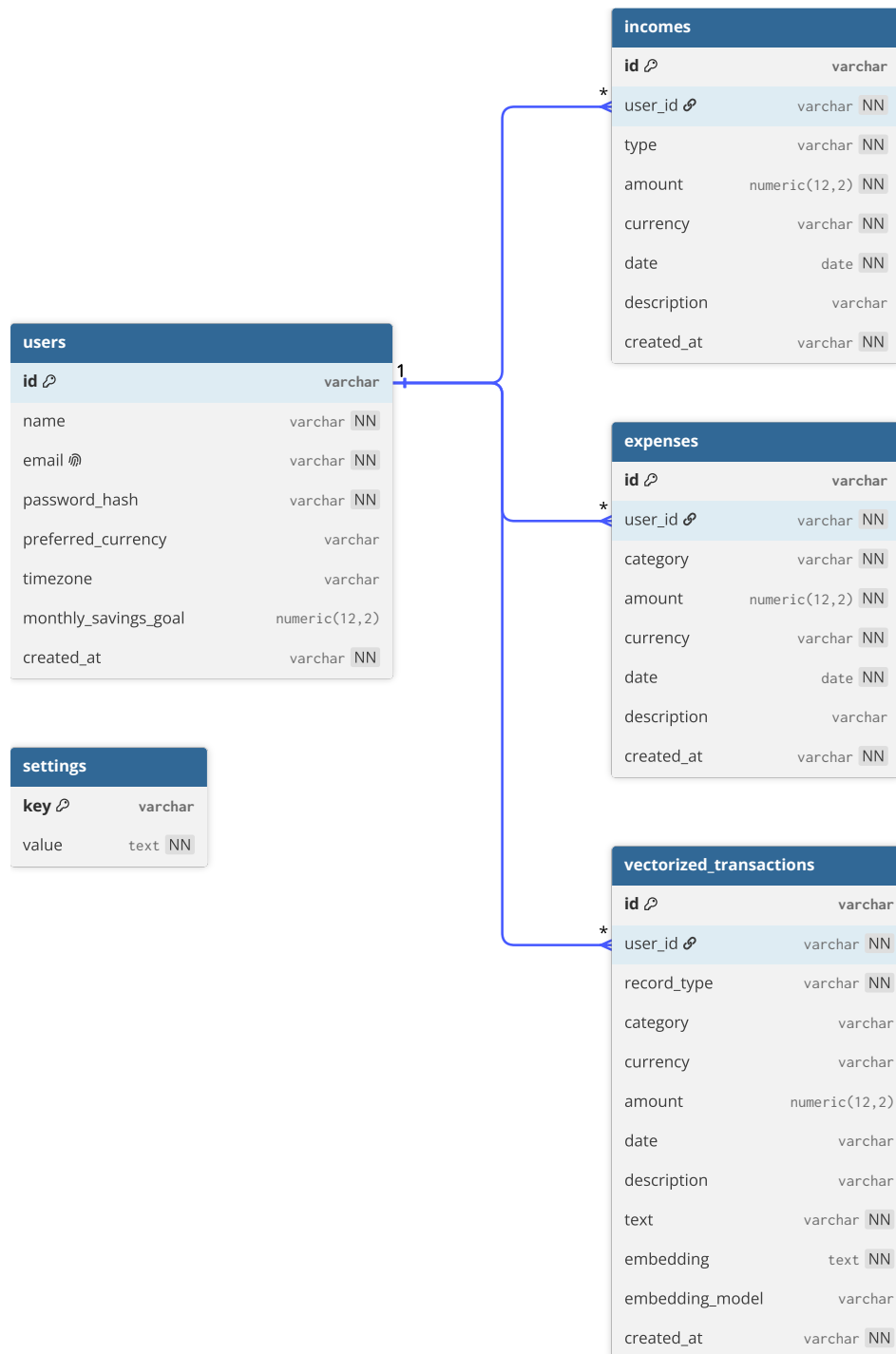


Figura 1: Diagrama Entidad-Relación de las bases de datos (elaborado con dbdiagram.io).

8.2. Selección del motor de base de datos: SQLite frente a PostgreSQL

PostgreSQL, preferentemente en Amazon RDS, constituye la alternativa convencional para entornos de producción con múltiples instancias, dado que permite compartir el estado a

través de la red. No obstante, la solución propuesta se ejecuta sobre **una única instancia EC2**, por lo que no existe el problema de compartir estado entre servidores y SQLite resulta suficiente. Además, SQLite no requiere un servidor de base de datos independiente, simplifica el despliegue y no genera costo adicional. La migración a PostgreSQL se justificaría si la solución escalara a cientos o miles de usuarios concurrentes.

9. Empaquetado con Docker

Cada microservicio y el Frontend se empaquetan como imágenes Docker independientes. Los Dockerfiles parten de imágenes base oficiales de **Docker Hub** (python:3-slim y node:20-alpine).

```
FROM python:3-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN addgroup --system appgroup && adduser --system --ingroup appgroup
    appuser \
    && mkdir -p /app/data && chown -R appuser:appgroup /app
USER appuser
EXPOSE 8001
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://
    localhost:8001/health')" || exit 1
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8001"]
```

Código 2: Dockerfile típico de un microservicio FastAPI.

```
services:
  user-service:    { build: ./user-service,    ports: ["8001:8001"],
                    volumes: ["users_data:/app/data"] }
  income-service: { build: ./income-service,  ports: ["8002:8002"],
                    volumes: ["incomes_data:/app/data"] }
  expense-service: { build: ./expense-service, ports: ["8003:8003"],
                    volumes: ["expenses_data:/app/data"] }
  ai-service:      { build: ./ai-service,      ports: ["8004:8004"],
                    volumes: ["vectors_data:/app/data"] }
  nginx:           { image: nginx:alpine,      ports: ["80:80"],
                    depends_on: [user-service, income-service, expense-service, ai-service]
                    }
volumes: { users_data: {}, incomes_data: {}, expenses_data: {},
```

```
vectors_data: {} }
```

Código 3: docker-compose.yml (resumen).

Persistencia: los contenedores son efímeros; cada servicio monta un **volumen** sobre /app/data (EBS), de modo que el archivo .db sobrevive a reinicios o redeploy.

10. Diagrama de Arquitectura de Solución en AWS

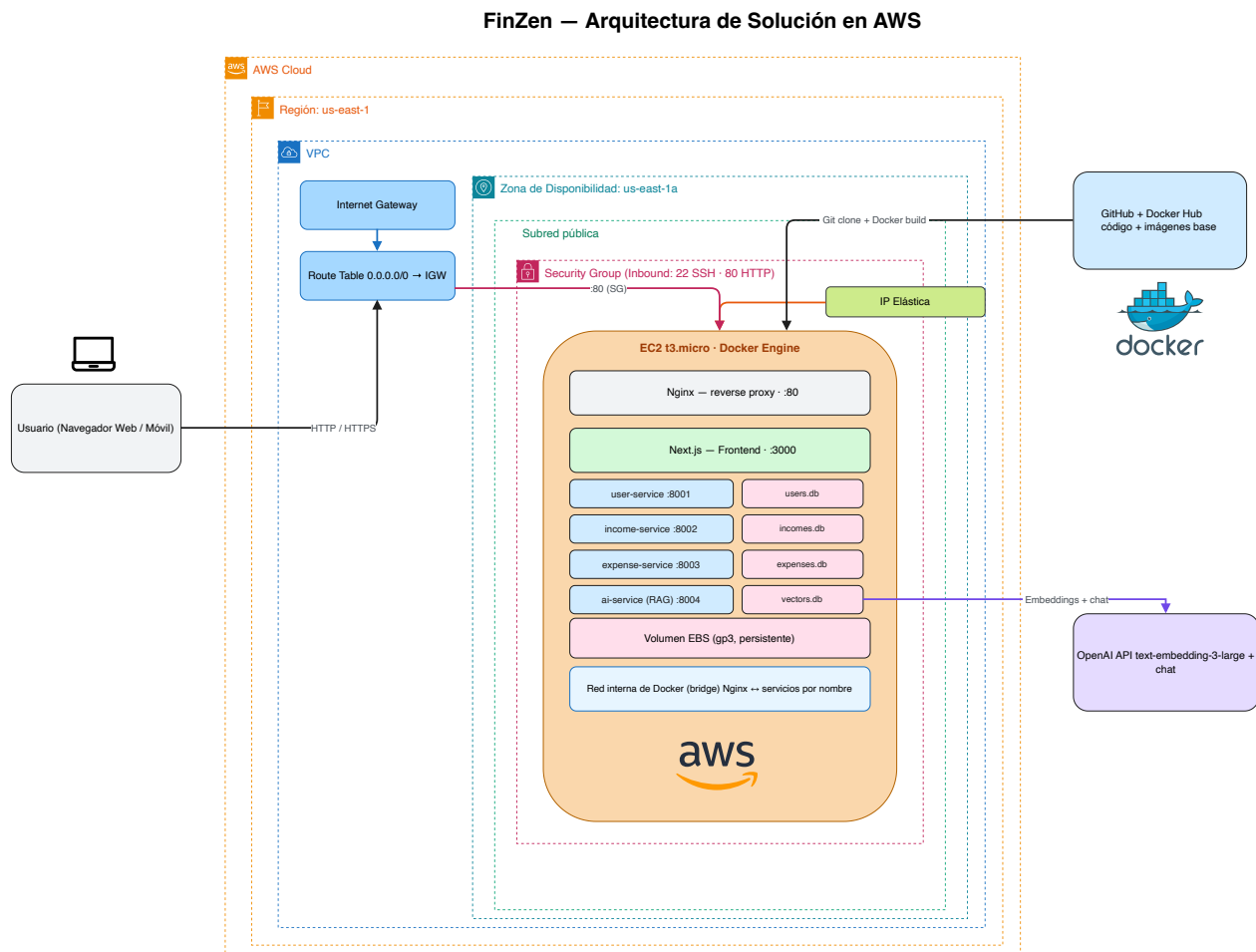


Figura 2: Arquitectura de FinZen en AWS (una instancia EC2 con IP elástica).

La solución emplea **una única instancia EC2** como servicio de cómputo y prescinde de Application Load Balancer, Auto Scaling, CloudFront, S3 y CloudWatch, en concordancia con el alcance mínimo del proyecto:

- Una instancia **EC2 t3.micro** en **us-east-1** con **IP elástica**: punto de entrada único.

- Sobre ella corre **Docker Engine** y, vía **Docker Compose**, 6 contenedores (Nginx, Next.js y los 4 FastAPI), comunicados por la red interna; Nginx publica todo en el puerto 80.
- El **User Data** clona los repositorios, construye las imágenes (`docker build`) y levanta los contenedores.
- Las bases SQLite viven en **volúmenes Docker** sobre el volumen **EBS**. El `ai-service` llama a la API de OpenAI.
- La `OPENAI_API_KEY` se inyecta como variable de entorno, sin Secrets Manager (arquitectura mínima solicitada).
- **IP fija:** se asigna una **IP elástica** para que la dirección no cambie al reiniciar la instancia.

10.1. Red en AWS (VPC, Internet Gateway, subred y Security Group)

La solución se despliega en la **VPC por defecto**, que ya incluye:

- **Internet Gateway (IGW)** adjunto a la VPC: conecta la VPC con internet.
- **Route Table** con la ruta `0.0.0.0/0 → Internet Gateway`.
- **Subredes públicas** en una **Zona de Disponibilidad** de `us-east-1`.
- **Security Group** (firewall virtual de la instancia) con reglas de entrada: 22 (SSH, para EC2 Instance Connect) y 80 (HTTP, por donde entra todo el tráfico web); y salida *All traffic* para `docker pull`, `git clone` y la API de OpenAI.

El **Internet Gateway** proporciona conectividad a la instancia en ambos sentidos: **entrante** (los usuarios acceden por el puerto 80) y **saliente** (descarga de imágenes de Docker Hub, clonado desde GitHub y llamadas a OpenAI). No se emplea **NAT Gateway**, ya que este sólo aplica a subredes privadas y añade costo; dado que se trata de una subred pública con IP elástica, el Internet Gateway constituye la alternativa técnica y económicamente adecuada.

11. Selección de región AWS

Región elegida: `us-east-1` (Norte de Virginia).

Justificación: se seleccionó por ser la región con mayor disponibilidad de servicios y con los costos más competitivos de EC2 y EBS, factor determinante para un proyecto académico. Asimismo, dado que la solución depende de una API externa (OpenAI) con buena conectividad desde Estados Unidos, la latencia adicional respecto de una región más cercana a Perú (`sa-east-1`) resulta marginal en comparación con el ahorro de costos.

12. Costos mensuales estimados en AWS

Estimación con la Calculadora de AWS ([estimado compartible del proyecto](#)) para una sola instancia EC2:

Servicio	Configuración	Costo/mes (USD)
EC2	1x t3.micro on-demand, Linux, 24/7	\$7.59
EBS	gp3, 20 GB (imágenes Docker, SQLite y código)	\$1.60
IP elástica (IPv4)	1 dirección elástica ($\approx \$0.005/\text{h}$, se cobre o no)	\$3.65
Transferencia de datos	≈ 5 GB/mes	\$0.45
TOTAL AWS (estimado)		$\approx \\$13.29$ / mes

Cuadro 5: Costos mensuales estimados.

Consideración de costos: desde febrero de 2024, AWS factura **toda dirección IPv4 pública** ($\approx \$0.005$ por hora), con independencia de que esté asociada o no a una instancia; en consecuencia, la **IP elástica** representa $\approx \$3.65/\text{mes}$. No obstante, al operar con una única instancia se evita el costo de un Application Load Balancer ($\approx \$16/\text{mes}$) y de una segunda instancia EC2.

El consumo de la API de OpenAI (embeddings y modelo de chat) se factura de forma independiente, fuera de AWS.

Segun la Calculadora de AWS, este estimado adjunta un costo mensual de \$13.29 USD y un costo total para 12 meses de 159.48 USD, considerando los costos sin cargo (VPC: \$0.00 USD).

12.1. Costos de OpenAI (aparte de AWS)

Precios oficiales de la API (fuente: <https://platform.openai.com/docs/pricing>), por 1 millón de tokens (plan Standard).

Modelo de embeddings	Precio / 1M tokens
text-embedding-3-large (usado por FinZen)	\$0.13
text-embedding-3-small	\$0.02

Cuadro 6: Precios de embeddings de OpenAI.

Modelo de chat	Entrada / 1M	Salida / 1M
gpt-4o-mini (usado por FinZen)	\$0.15	\$0.60
gpt-4.1-nano	\$0.10	\$0.40
gpt-4.1-mini	\$0.40	\$1.60
gpt-4o	\$2.50	\$10.00
gpt-3.5-turbo	\$0.50	\$1.50

Cuadro 7: Precios de modelos de chat de OpenAI.

Para el volumen estimado del proyecto (1000 movimientos vectorizados y 200 consultas mensuales), el costo de IA asciende a aproximadamente **\$0.04/mes** (embeddings `text-embedding-3-large` y chat `gpt-4o-mini`), un valor marginal en relación con el costo de AWS. Cabe señalar que existen modalidades más económicas: procesamiento por lotes (*Batch*, $\approx 50\%$ menor) y caché de entrada ($\approx 90\%$ menor en tokens repetidos).

Parte C: Implementación del MVP

13. Catálogo de APIs

Todos los servicios exponen respuestas en formato JSON y documentan su interfaz mediante OpenAPI/Swagger (/docs). Nginx publica cada microservicio bajo /api/*. Los endpoints protegidos requieren `Authorization: Bearer <token>`.

Servicio	Método	Ruta pública (Nginx)	Auth	Descripción
user	POST	/api/users/users	No	Registrar usuario
user	POST	/api/users/auth/login	No	Login (emite JWT)
user	GET	/api/users/users/{id}	Sí	Perfil
user	PUT	/api/users/users/{id}	Sí	Actualizar perfil
income	POST	/api/incomes/incomes	Sí	Registrar ingreso
income	GET	/api/incomes/incomes/{user_id}	Sí	Listar ingresos
income	GET	/api/incomes/incomes/detail/{id}	Sí	Detalle
income	DELETE	/api/incomes/incomes/{id}	Sí	Eliminar
expense	POST	/api/expenses/expenses	Sí	Registrar gasto
expense	GET	/api/expenses/expenses/{user_id}	Sí	Listar (filtros)
expense	GET	/api/expenses/expenses/detail/{id}	Sí	Detalle
expense	GET	/api/expenses/expenses/categories/list	Sí	Categorías
expense	DELETE	/api/expenses/expenses/{id}	Sí	Eliminar
ai	POST	/api/ai/query	Sí	Consulta en lenguaje natural (RAG)
ai	POST	/api/ai/reindex	Sí	Re-indexar movimientos
ai	GET/ PUT/ DELETE	/api/ai/settings/openai-key	Sí	Clave de OpenAI

14. Implementación del Backend

El backend se compone de 4 microservicios en Python (FastAPI), cada uno empaquetado como contenedor Docker con su propia base SQLite persistida en un volumen y orquestados mediante Docker Compose. Los endpoints protegidos emplean JWT. Cada servicio expone `/health` e implementa prácticas de calidad y seguridad: CORS restringido, validación con Pydantic, ejecución con usuario no privilegiado y `HEALTHCHECK` en Docker.

14.1. Configuración de la clave de OpenAI

El asistente (HU7) usa la API de OpenAI para los *embeddings* y el modelo de chat. La clave se obtiene en el panel de OpenAI (<https://platform.openai.com/api-keys>): *API keys* → *Create new secret key*. Luego se configura de una de estas dos formas:

- **En el despliegue:** como variable `OPENAI_API_KEY` en el archivo `.env` del backend.
- **En tiempo de ejecución:** desde la interfaz web (*Mi Perfil*), que la guarda en la base SQLite del `ai-service` (nunca en el navegador).

Si no se configura, el `ai-service` funciona en **modo local** (embeddings deterministas) para poder probar sin conexión.

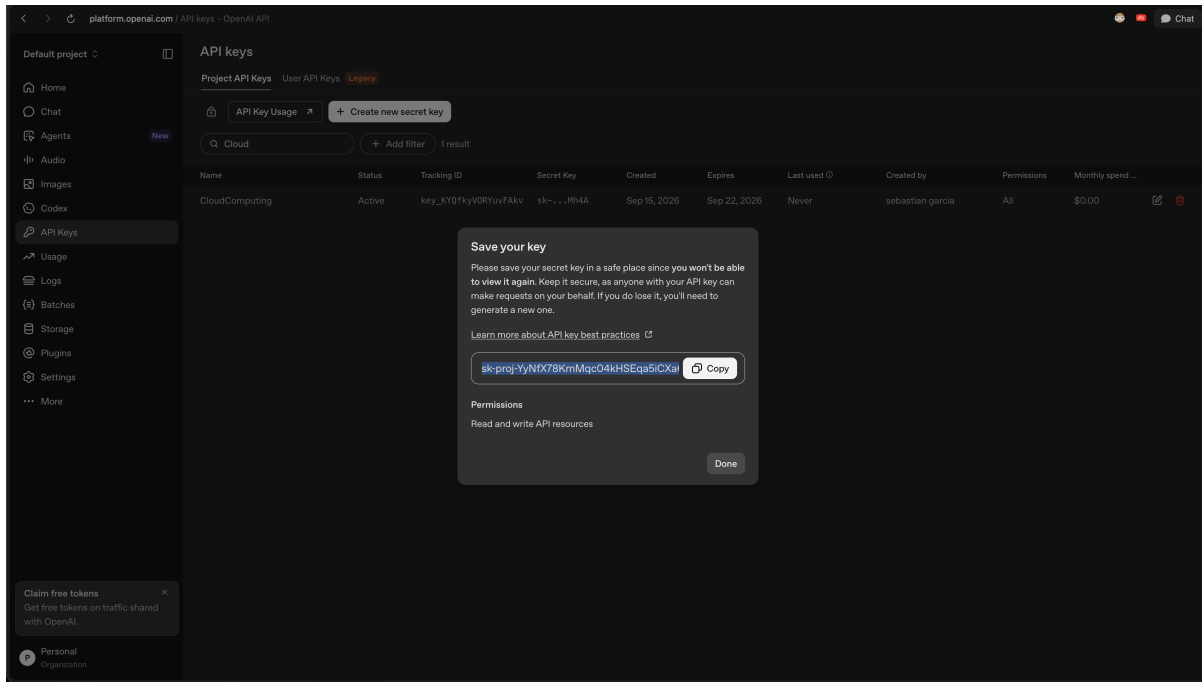


Figura 3: Panel de OpenAI (platform.openai.com/api-keys): creación de una nueva clave secreta de API.

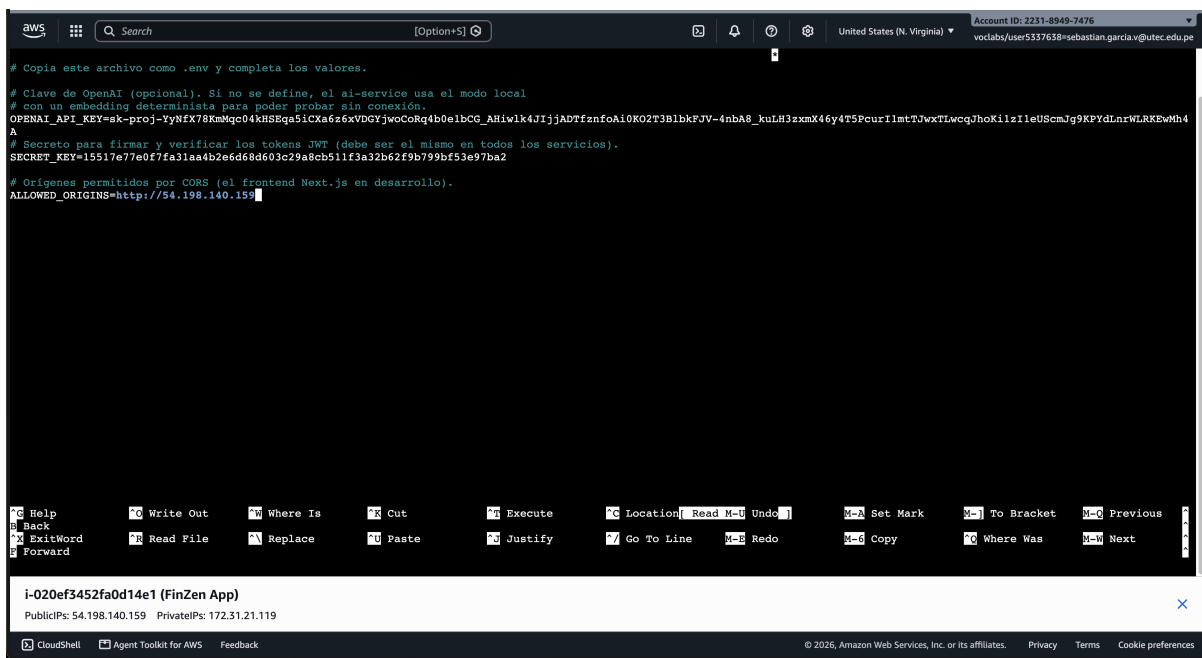


Figura 4: Archivo .env del backend en la EC2, con OPENAI_API_KEY, SECRET_KEY y ALLOWED_ORIGINS configurados.

15. Evidencia de uso con Postman

La colección importable, junto con los entornos Local y AWS y la guía de evidencias, se encuentra en `postman/` del repositorio de documentación [finzen-docs](#). A continuación se presentan las capturas de las peticiones ejecutadas con **Newman** (CLI oficial de Postman) contra el backend en ejecución (registro, inicio de sesión, registro de ingresos y gastos, filtrado, las tres consultas al asistente y la reindexación), con sus códigos de estado reales.

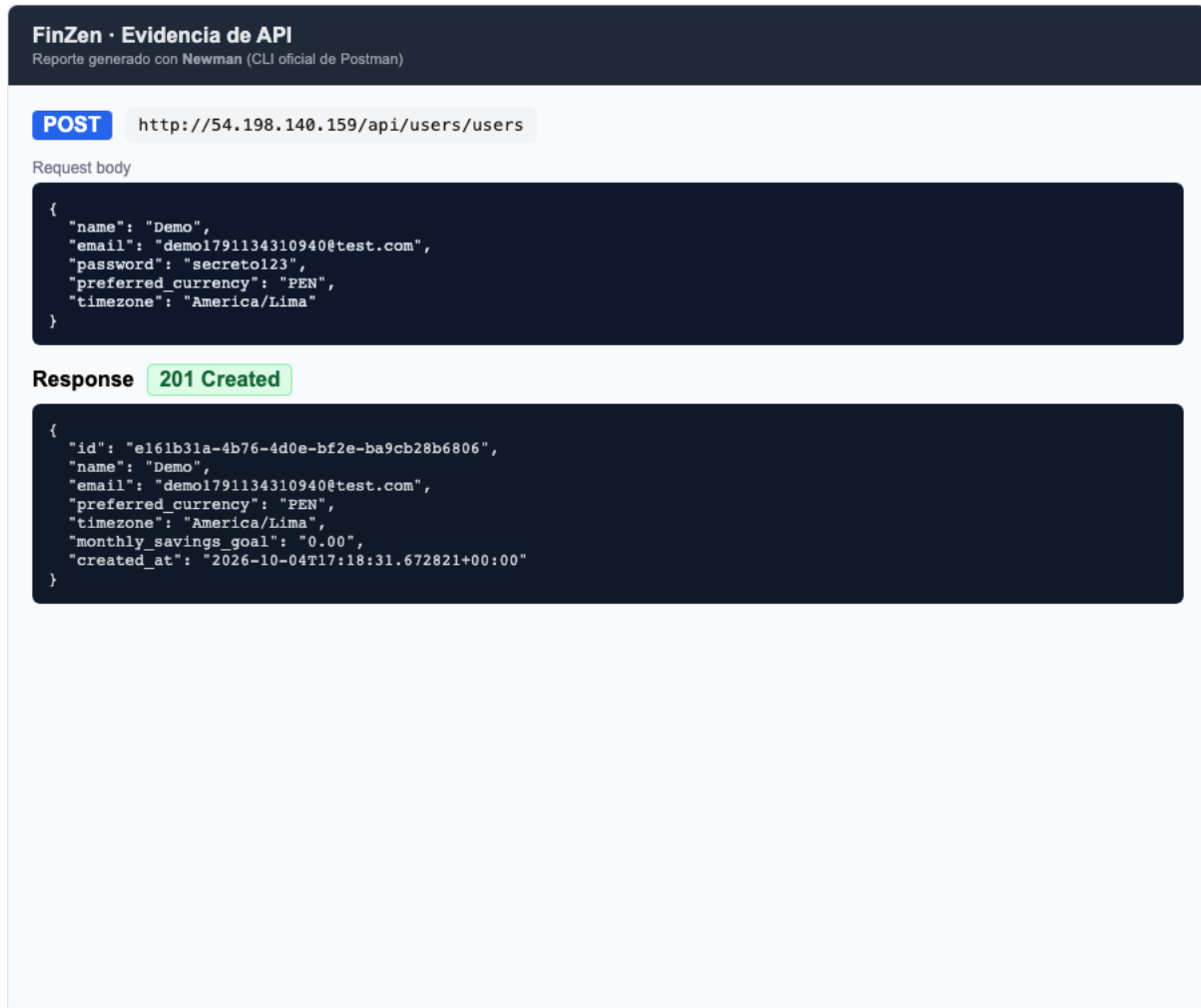


Figura 5: Petición de registro: POST `/api/users/users` con el cuerpo JSON y respuesta 201 Created con el usuario creado.

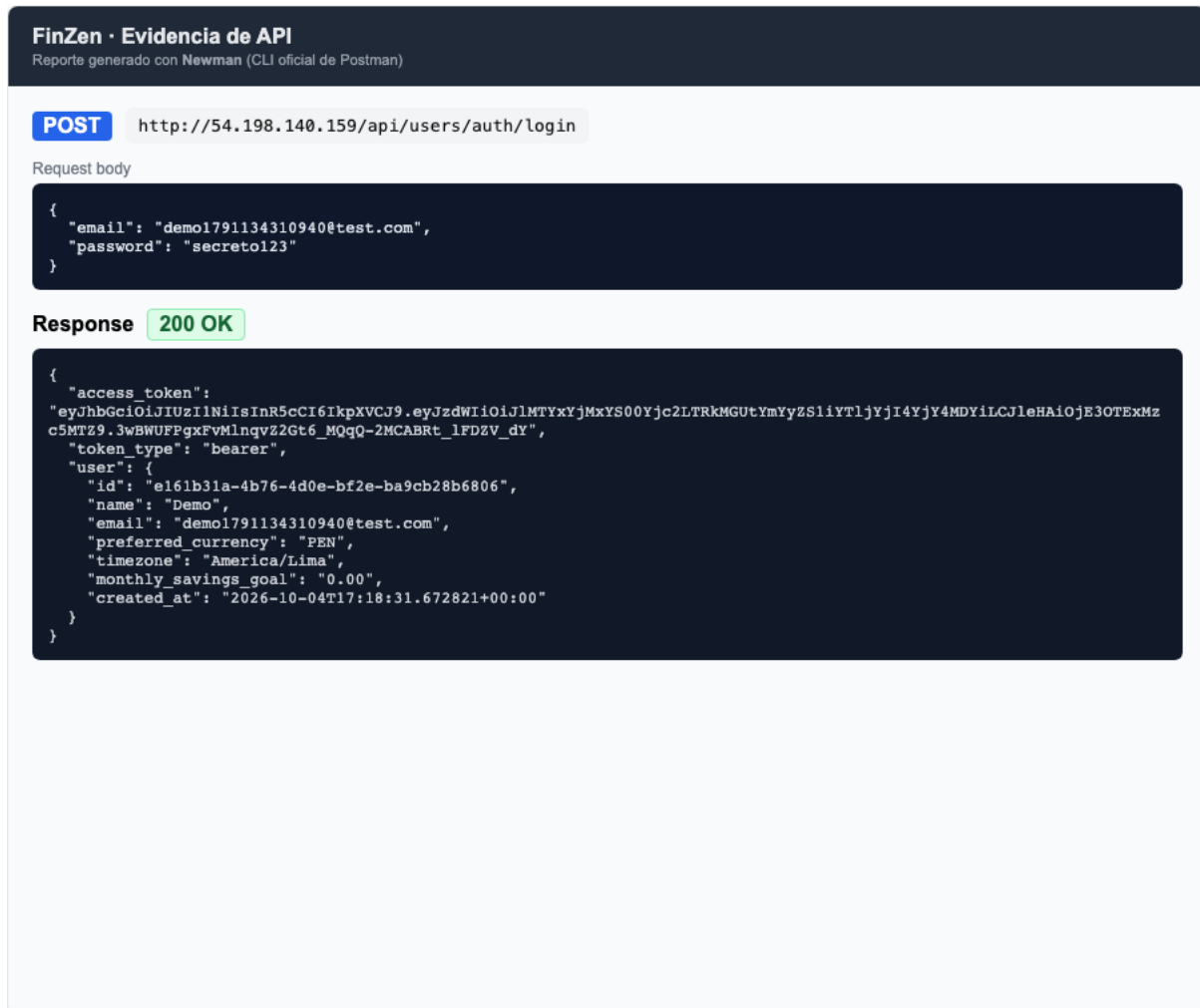


Figura 6: Petición de login: POST /api/users/auth/login que devuelve 200 OK con el access_token (JWT).

FinZen · Evidencia de API
Reporte generado con Newman (CLI oficial de Postman)

POST http://54.198.140.159/api/incomes/incomes

Request body

```
{
  "user_id": "e161b31a-4b76-4d0e-bf2e-ba9cb28b6806",
  "type": "Sueldo",
  "amount": "3500.50",
  "currency": "PEN",
  "date": "2026-09-01",
  "description": "Sueldo de setiembre"
}
```

Response **201 Created**

```
{
  "id": "0ad3d20a-1583-48dc-bbad-546a6696cf35",
  "user_id": "e161b31a-4b76-4d0e-bf2e-ba9cb28b6806",
  "type": "Sueldo",
  "amount": "3500.50",
  "currency": "PEN",
  "date": "2026-09-01",
  "description": "Sueldo de setiembre",
  "created_at": "2026-10-04T17:18:47.769052+00:00"
}
```

Figura 7: Registro de un ingreso: POST /api/incomes/incomes con respuesta 201 Created.

FinZen · Evidencia de API
Reporte generado con Newman (CLI oficial de Postman)

POST http://54.198.140.159/api/expenses/expenses

Request body

```
{
  "user_id": "e161b31a-4b76-4d0e-bf2e-ba9cb28b6806",
  "category": "Alimentacion",
  "amount": "450.50",
  "currency": "PEN",
  "date": "2026-09-05",
  "description": "Supermercado"
}
```

Response **201 Created**

```
{
  "id": "b7d605fb-1c3d-4cd5-ab5c-6eb57e35c6c9",
  "user_id": "e161b31a-4b76-4d0e-bf2e-ba9cb28b6806",
  "category": "Alimentacion",
  "amount": "450.50",
  "currency": "PEN",
  "date": "2026-09-05",
  "description": "Supermercado",
  "created_at": "2026-10-04T17:19:00.696080+00:00"
}
```

Figura 8: Registro de un gasto en soles (PEN): POST /api/expenses/expenses con respuesta 201 Created.

FinZen · Evidencia de API
Reporte generado con Newman (CLI oficial de Postman)

POST http://54.198.140.159/api/expenses/expenses

Request body

```
{
  "user_id": "e161b31a-4b76-4d0e-bf2e-ba9cb28b6806",
  "category": "Entretenimiento",
  "amount": "20.00",
  "currency": "USD",
  "date": "2026-09-06",
  "description": "Netflix"
}
```

Response **201 Created**

```
{
  "id": "e082b7b8-f547-48b0-89fa-43350df3f949",
  "user_id": "e161b31a-4b76-4d0e-bf2e-ba9cb28b6806",
  "category": "Entretenimiento",
  "amount": "20.00",
  "currency": "USD",
  "date": "2026-09-06",
  "description": "Netflix",
  "created_at": "2026-10-04T17:19:06.625776+00:00"
}
```

Figura 9: Registro de un gasto en dólares (USD): misma ruta, respuesta 201 Created.

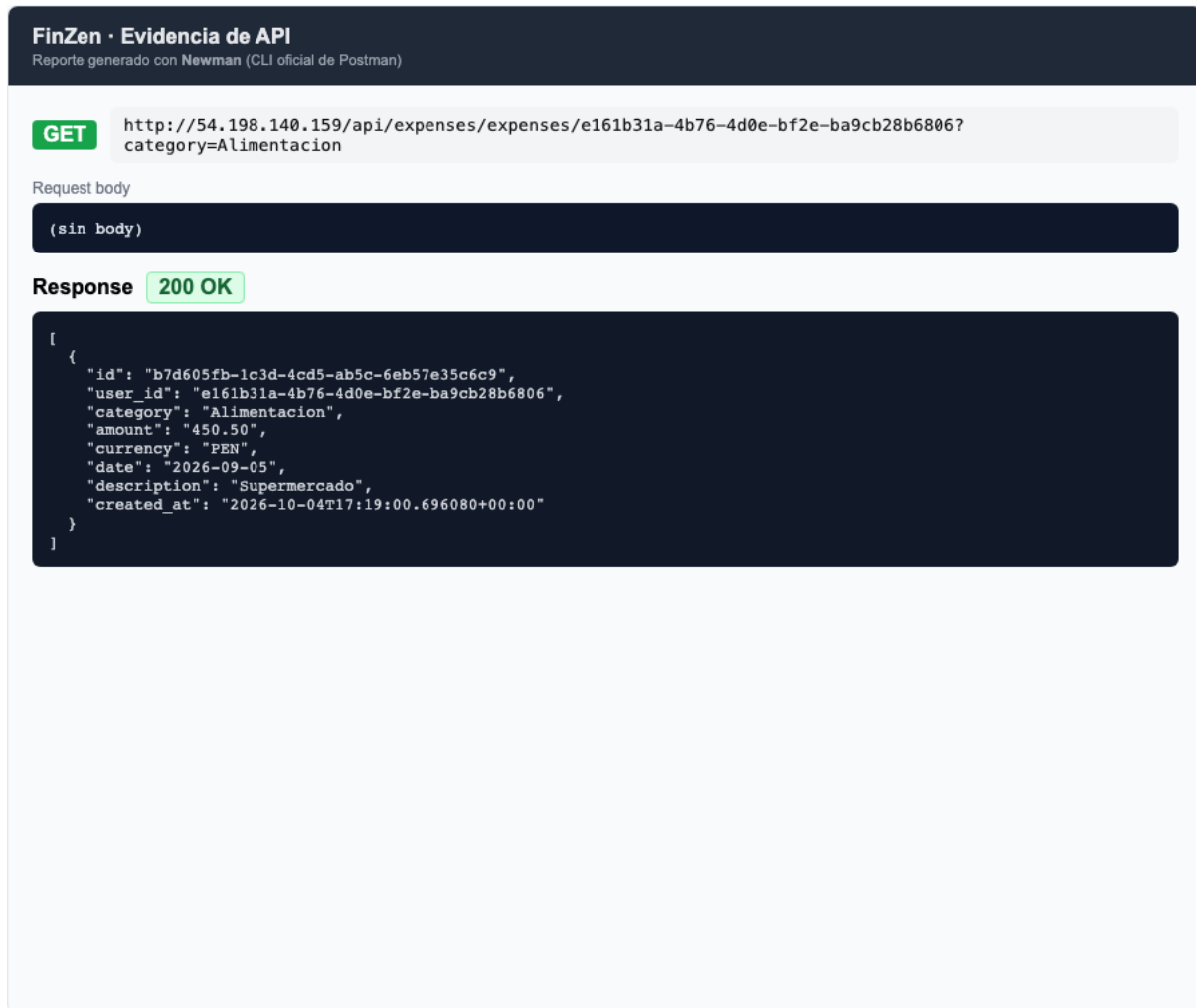


Figura 10: Listado de gastos filtrado por categoría: GET `/api/expenses/expenses/{user_id}?category=Alimentacion` con respuesta 200 OK.

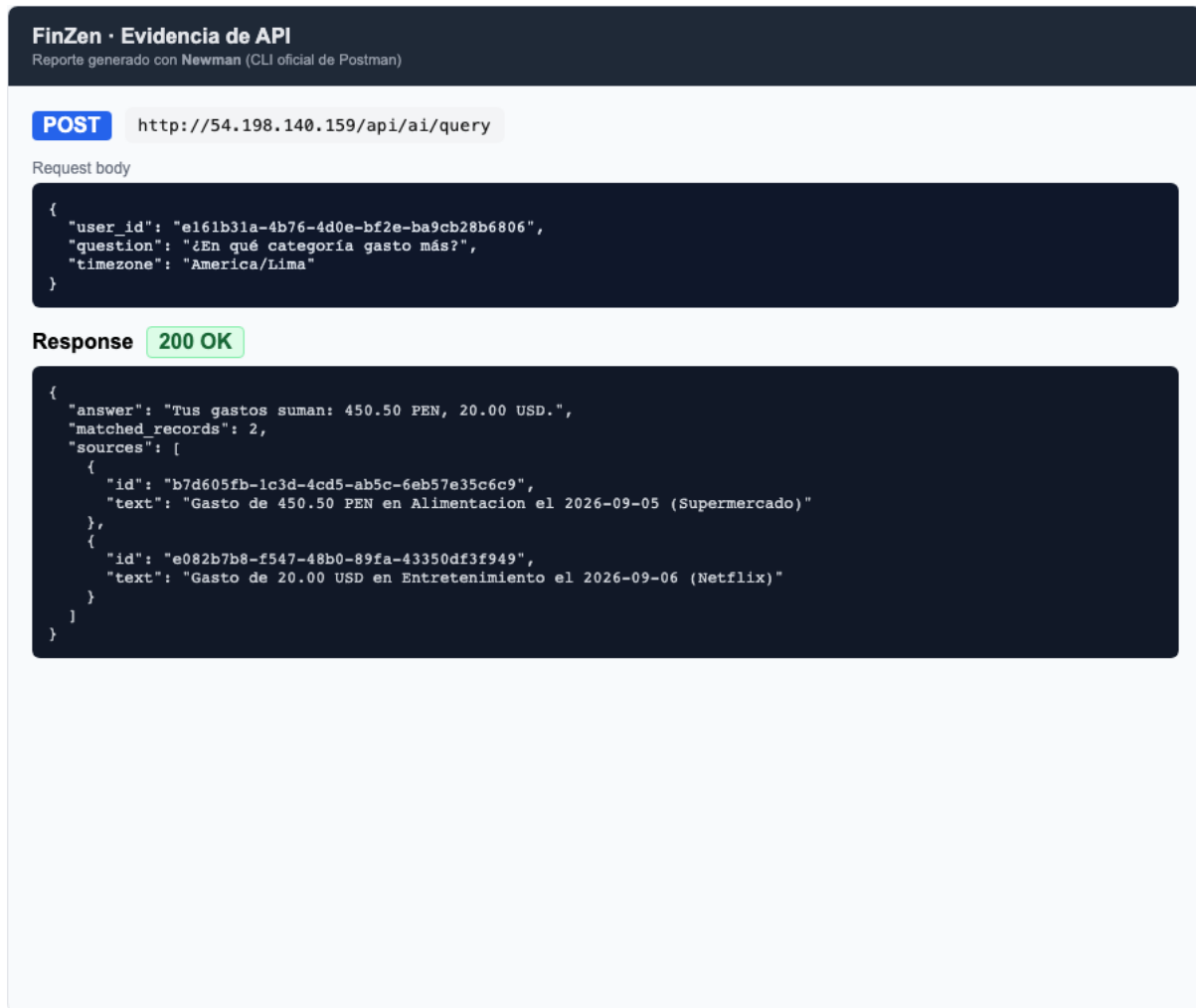


Figura 11: Consulta al asistente 1: “¿en qué categoría gasto más?” (POST /api/ai/query) con respuesta en lenguaje natural.

FinZen · Evidencia de API
Reporte generado con Newman (CLI oficial de Postman)

POST http://54.198.140.159/api/ai/query

Request body

```
{
  "user_id": "e161b31a-4b76-4d0e-bf2e-ba9cb28b6806",
  "question": "¿Cuánto he gastado este mes?",
  "timezone": "America/Lima"
}
```

Response **200 OK**

```
{
  "answer": "Tus gastos suman: 450.50 PEN, 20.00 USD.",
  "matched_records": 3,
  "sources": [
    {
      "id": "0ad3d20a-1583-48dc-bbad-546a6696cf35",
      "text": "Ingreso de 3500.50 PEN en Sueldo el 2026-09-01 (Sueldo de setiembre)"
    },
    {
      "id": "b7d605fb-1c3d-4cd5-ab5c-6eb57e35c6c9",
      "text": "Gasto de 450.50 PEN en Alimentacion el 2026-09-05 (Supermercado)"
    },
    {
      "id": "e082b7b8-f547-48b0-89fa-43350df3f949",
      "text": "Gasto de 20.00 USD en Entretenimiento el 2026-09-06 (Netflix)"
    }
  ]
}
```

Figura 12: Consulta al asistente 2: “¿cuánto he gastado este mes?” con la suma de los gastos por moneda.

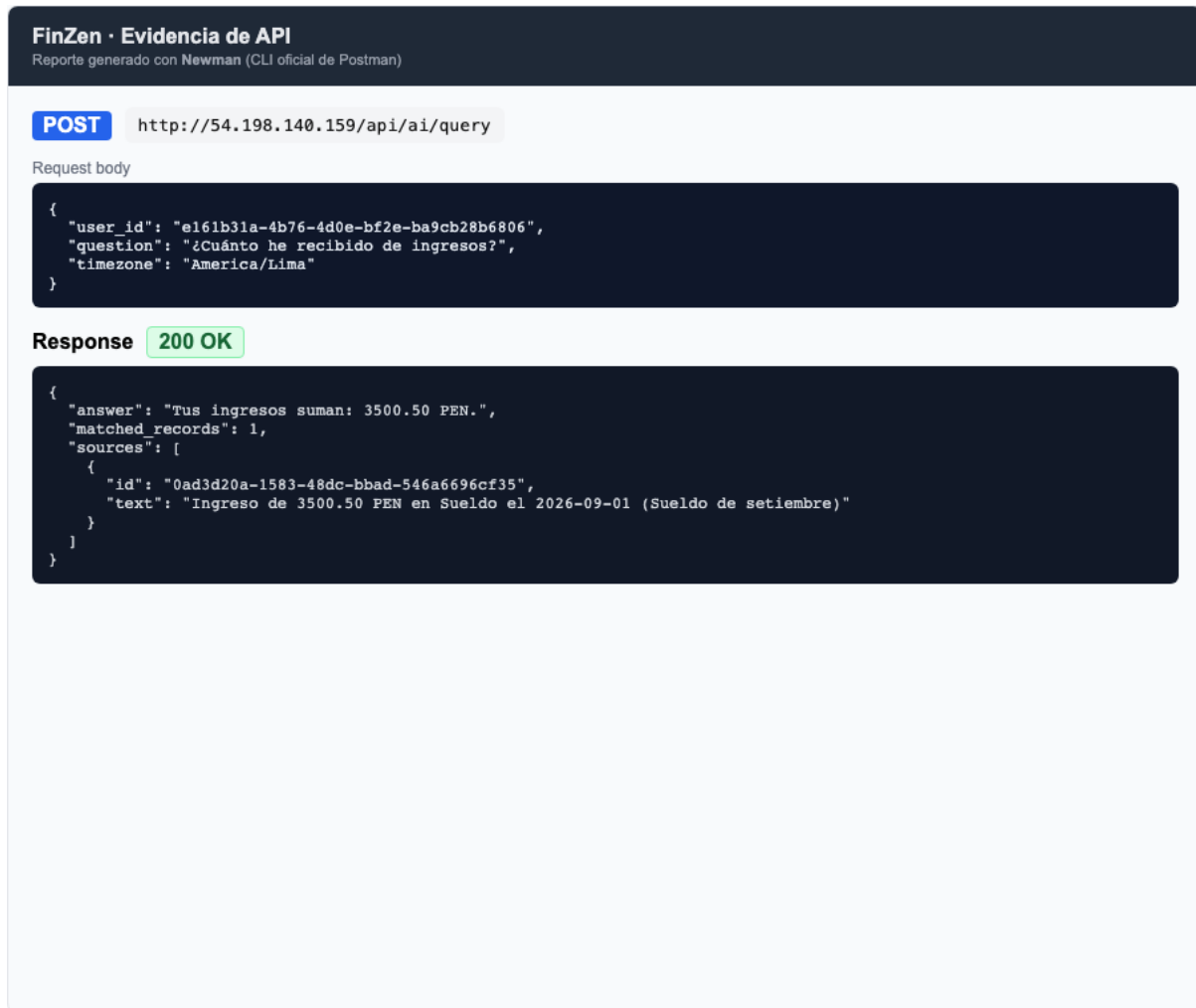


Figura 13: Consulta al asistente 3: “¿cuánto he recibido de ingresos?” con el total de ingresos.

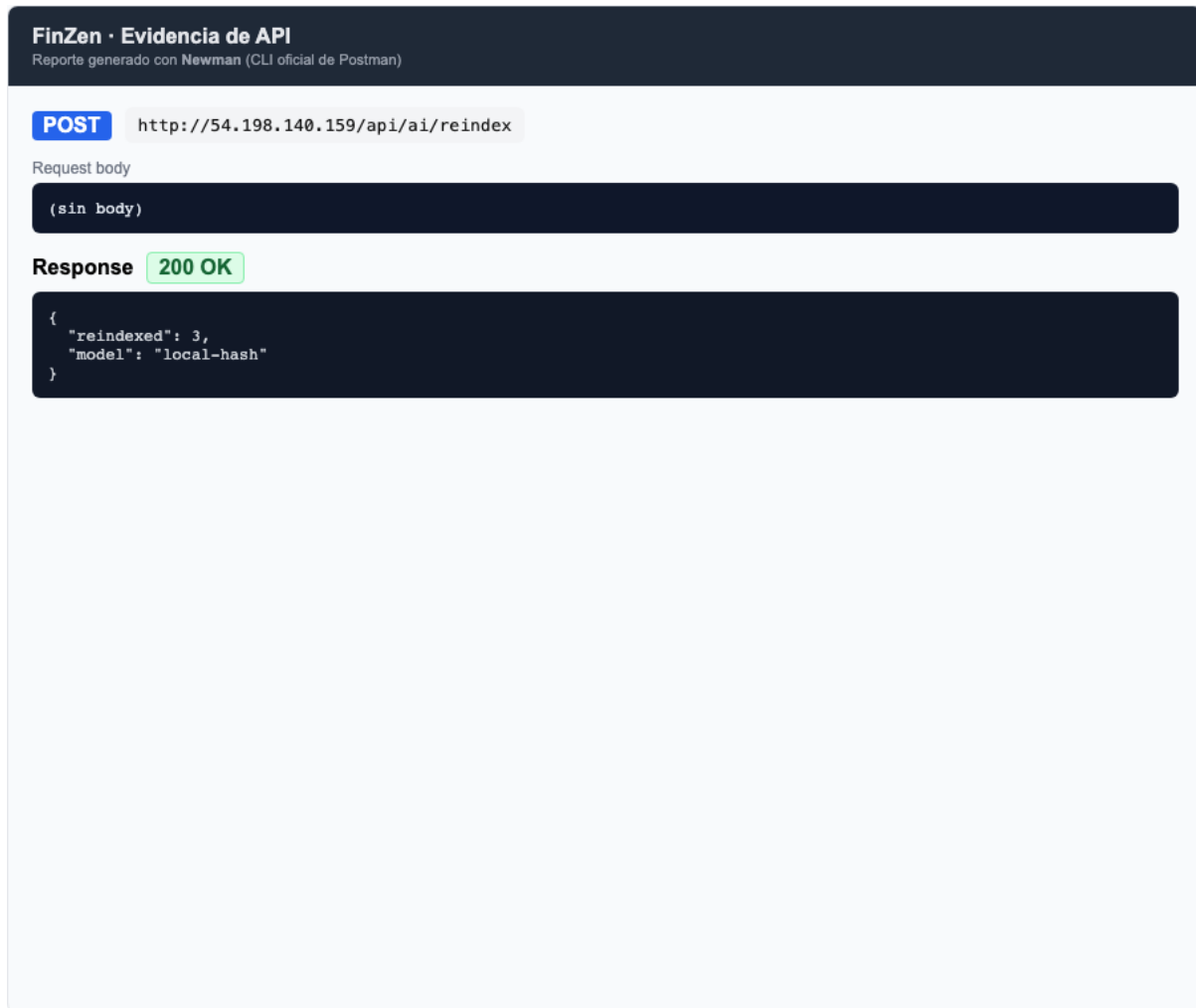


Figura 14: Re-indexación de movimientos: POST /api/ai/reindex regenera los vectores con el modelo de *embeddings*.

16. Frontend conectado (BFF + JWT)

El frontend Next.js consume las APIs a través de un **BFF**: el JWT se guarda en una cookie `httpOnly` y el servidor de Next lo inyecta al reenviar la petición a Nginx.

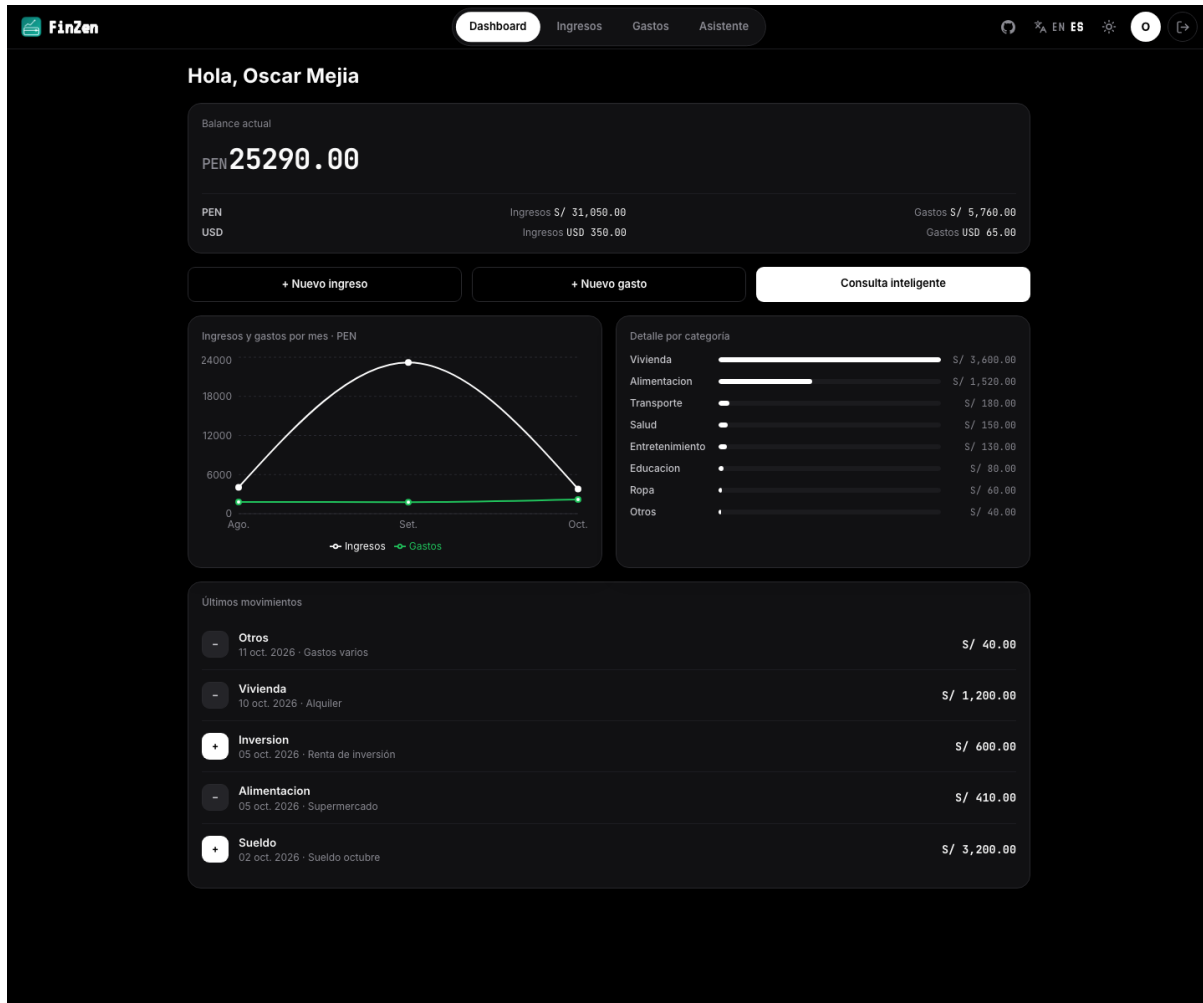


Figura 15: *Dashboard*: balance por moneda, gráfico de gastos por categoría, detalle por categoría y últimos movimientos.

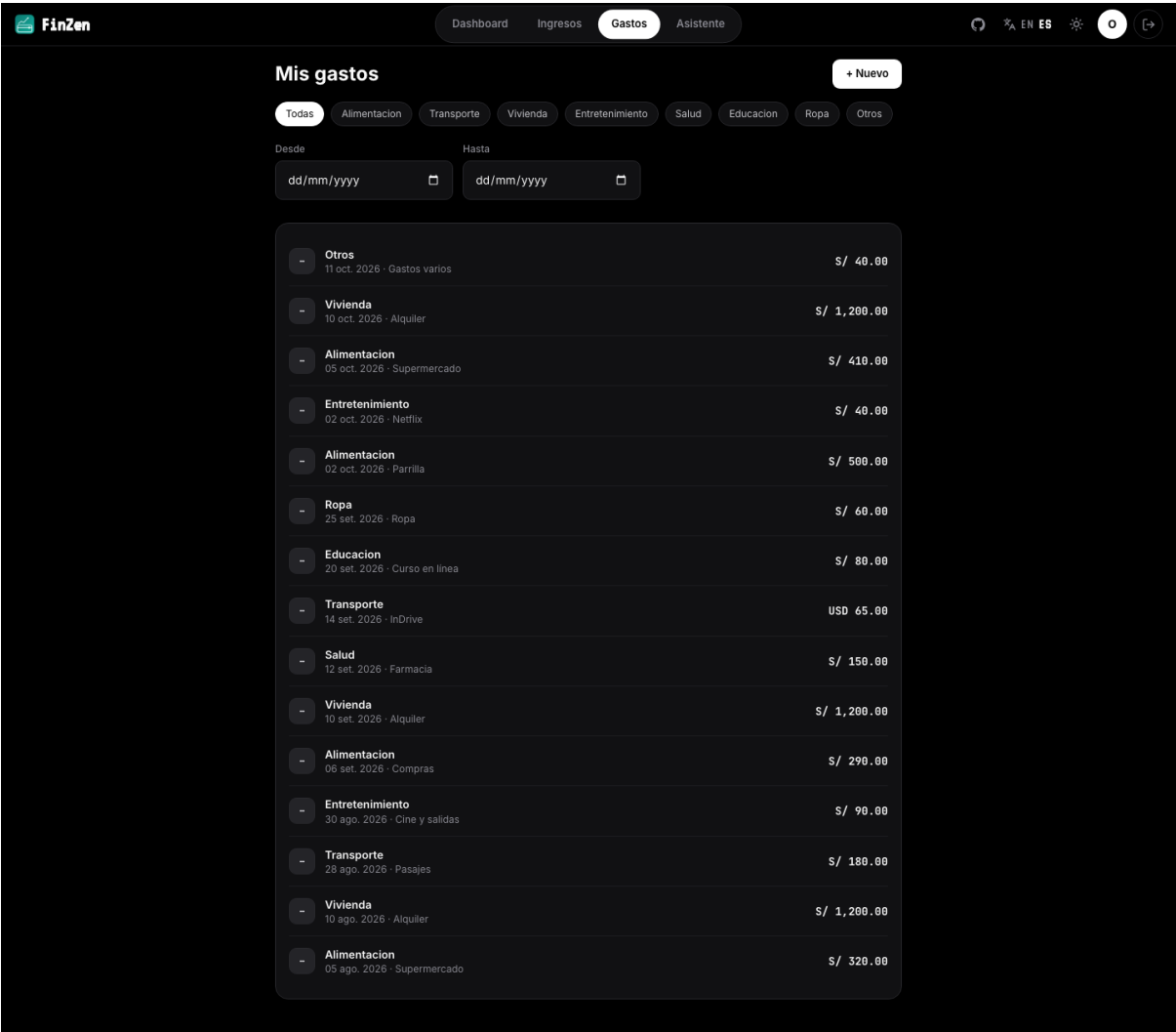


Figura 16: Listado de gastos con filtros por categoría y rango de fechas, y acción de eliminar.

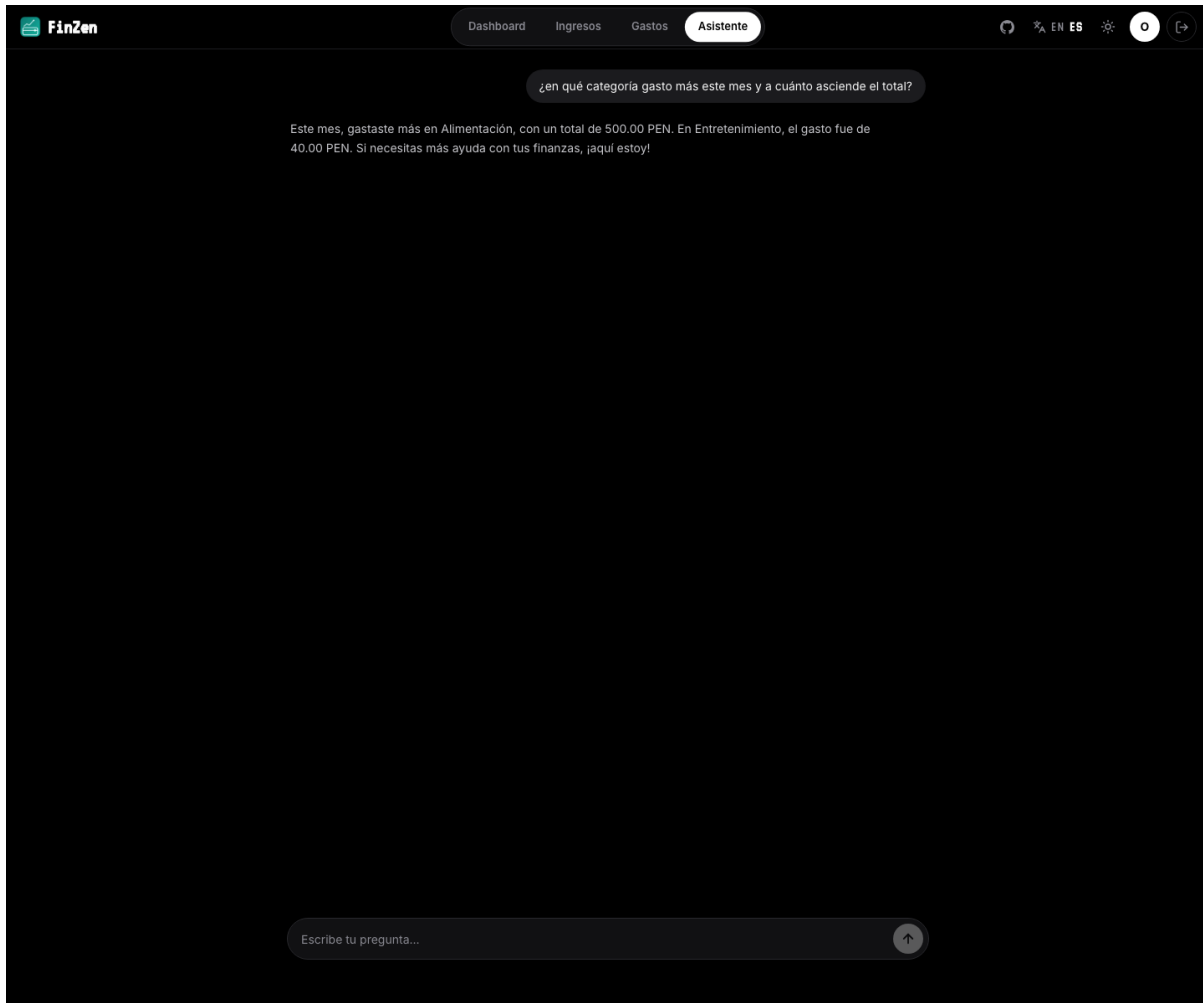


Figura 17: Asistente de IA respondiendo una consulta en lenguaje natural sobre las finanzas del usuario.

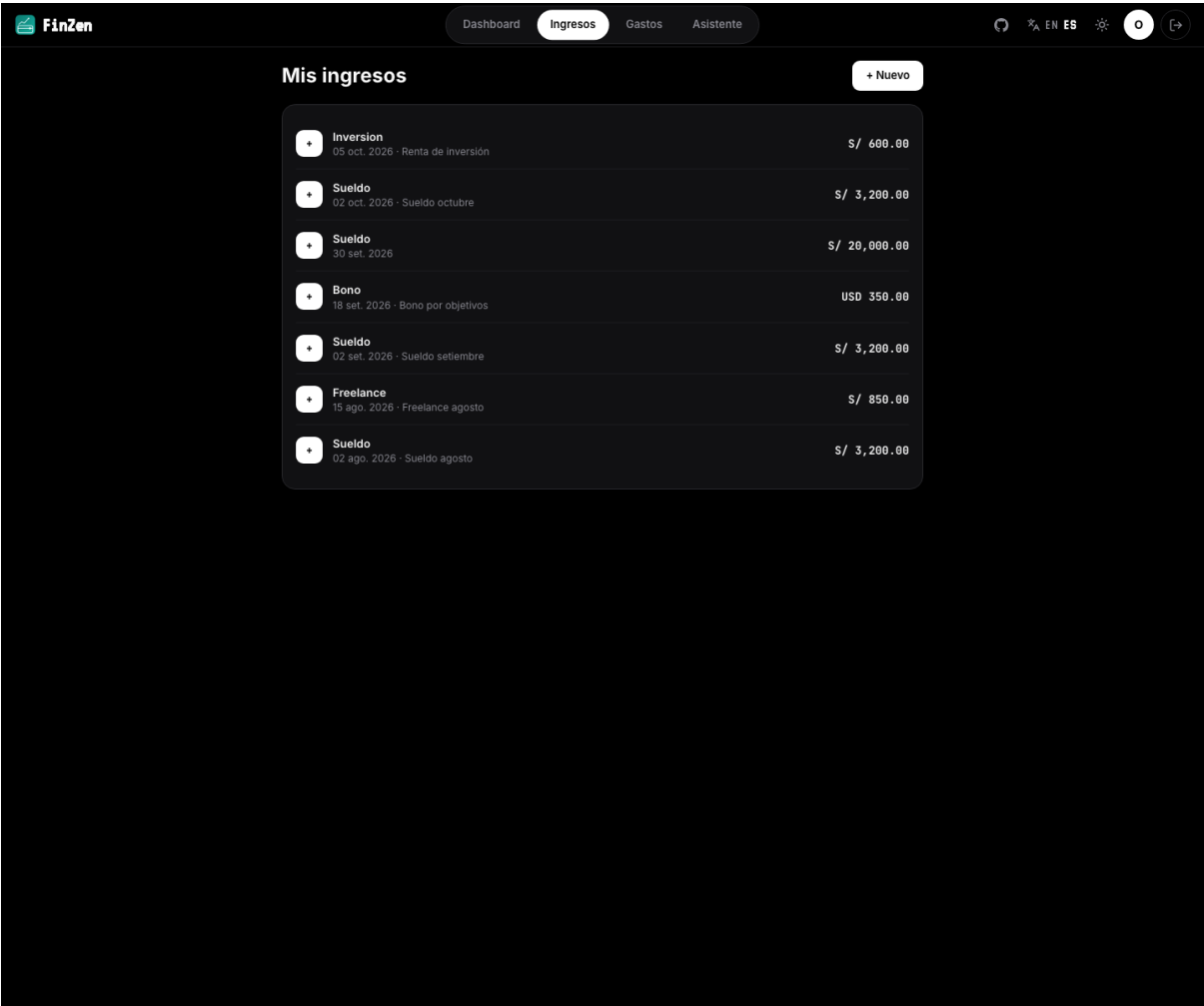


Figura 18: Listado de los ingresos registrados por el usuario.

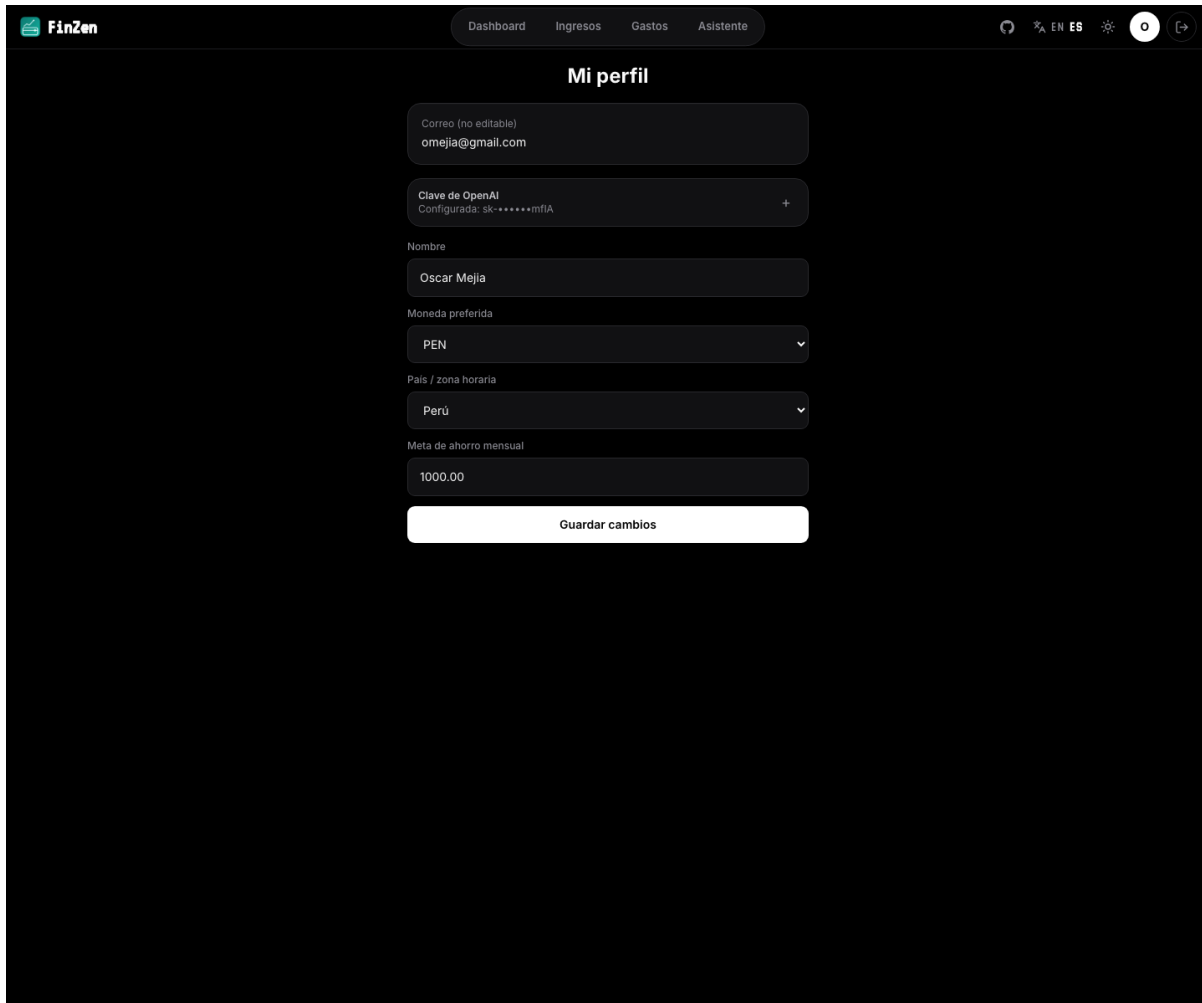


Figura 19: Perfil del usuario: datos personales, moneda y zona horaria configurados.

17. Despliegue en AWS

El despliegue se realiza sobre **una instancia EC2** con **IP elástica**, administrada mediante **EC2 Instance Connect** (sin gestión de llaves SSH). El script de *User Data* instala Docker, clona los repositorios y ejecuta `docker compose up -d -build`.

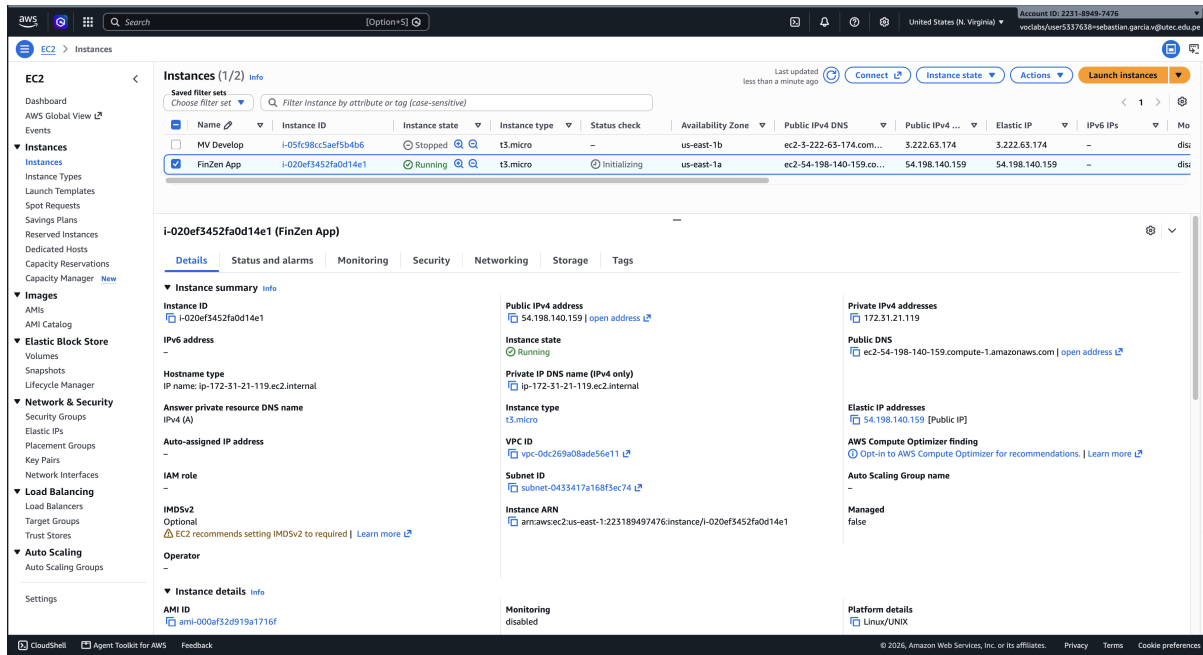


Figura 20: Consola de AWS (región us-east-1) con la instancia FinZen App en estado *Running*, tipo t3.micro y su IP elástica 54.198.140.159.

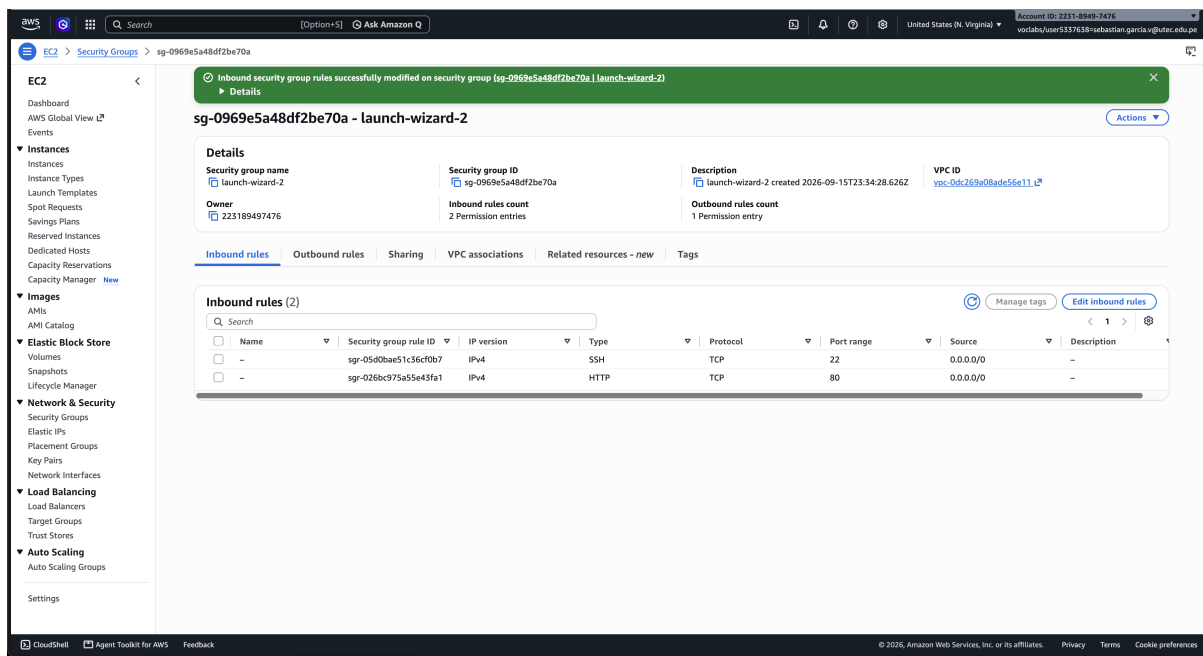


Figura 21: Reglas de *entrada* del Security Group: 22 (SSH) y 80 (HTTP), ambas desde 0.0.0.0/0.

```

i-020ef3452fa0d14e1 (FinZen App)
PublicIPs: 54.198.140.159 PrivateIPs: 172.31.21.119
CloudShell Agent Toolkit for AWS Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

~/finzen-app (main) $ docker compose ps
docker compose ps
docker ps
[+] up 5/5
  Container finzen-user-service Healthy 8.7s
  Container finzen-ai-service Healthy 8.7s
  Container finzen-expense-service Healthy 6.7s
  Container finzen-income-service Healthy 7.2s
  Container finzen-nginx Started 0.4s

NAME                IMAGE                COMMAND                SERVICE    CREATED     STATUS      PORTS
finzen-ai-service    finzen-app-ai-service    "uvicorn main:app --..."    ai-service  4 hours ago Up 15 seconds (healthy) 0.0.0.0:8004->8004/tcp, [::]:8004->8004/tcp
finzen-expense-service    finzen-app-expense-service    "uvicorn main:app --..."    expense-service  4 hours ago Up 8 seconds (healthy) 0.0.0.0:8003->8003/tcp, [::]:8003->8003/tcp
finzen-income-service    finzen-app-income-service    "uvicorn main:app --..."    income-service  4 hours ago Up 7 seconds (healthy) 0.0.0.0:8002->8002/tcp, [::]:8002->8002/tcp
finzen-nginx          nginx:alpine            "/docker-entrypoint..."    nginx         4 hours ago Up Less than a second 0.0.0.0:80->80/tcp, [::]:80->80/tcp
finzen-user-service    finzen-app-user-service    "uvicorn main:app --..."    user-service   4 hours ago Up 15 seconds (healthy) 0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp

CONTAINER ID   IMAGE                COMMAND                CREATED     STATUS      PORTS      NAMES
6cbdf884500f   finzen-webui         "/docker-entrypoint.s..."  7 minutes ago Up 7 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp finzen-webui
1685f4b3c7c0   nginx:alpine         "/docker-entrypoint..."  4 hours ago Up 1 second 0.0.0.0:80->80/tcp, [::]:80->80/tcp finzen-nginx
b775363bda9c   finzen-app-income-service    "uvicorn main:app --..."  4 hours ago Up 8 seconds (healthy) 0.0.0.0:8002->8002/tcp, [::]:8002->8002/tcp finzen-income-service
1d24daa14a7    finzen-app-expense-service    "uvicorn main:app --..."  4 hours ago Up 8 seconds (healthy) 0.0.0.0:8003->8003/tcp, [::]:8003->8003/tcp finzen-expense-service
5a279ada9a76   finzen-app-ai-service    "uvicorn main:app --..."  4 hours ago Up 15 seconds (healthy) 0.0.0.0:8004->8004/tcp, [::]:8004->8004/tcp finzen-ai-service
d880d47a59a5   finzen-app-user-service    "uvicorn main:app --..."  4 hours ago Up 16 seconds (healthy) 0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp finzen-user-service

~/finzen-app (main) $ docker ps
CONTAINER ID   IMAGE                COMMAND                CREATED     STATUS      PORTS      NAMES
6cbdf884500f   finzen-webui         "/docker-entrypoint.s..."  7 minutes ago Up 7 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp finzen-webui
1685f4b3c7c0   nginx:alpine         "/docker-entrypoint..."  4 hours ago Up 8 seconds (healthy) 0.0.0.0:80->80/tcp, [::]:80->80/tcp finzen-nginx
b775363bda9c   finzen-app-income-service    "uvicorn main:app --..."  4 hours ago Up 13 seconds (healthy) 0.0.0.0:8002->8002/tcp, [::]:8002->8002/tcp finzen-income-service
1d24daa14a7    finzen-app-expense-service    "uvicorn main:app --..."  4 hours ago Up 13 seconds (healthy) 0.0.0.0:8003->8003/tcp, [::]:8003->8003/tcp finzen-expense-service
5a279ada9a76   finzen-app-ai-service    "uvicorn main:app --..."  4 hours ago Up 20 seconds (healthy) 0.0.0.0:8004->8004/tcp, [::]:8004->8004/tcp finzen-ai-service
d880d47a59a5   finzen-app-user-service    "uvicorn main:app --..."  4 hours ago Up 21 seconds (healthy) 0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp finzen-user-service

~/finzen-app (main) $ curl -s -o /dev/null -w "%{http_code}\n" http://localhost/
200
curl -s http://localhost/api/ai/health
200
{"status":"ok","service":"ai-service","openai_enabled":true}~/finzen-app (main) $

```

Figura 22: Terminal de la EC2: `docker compose ps` y `docker ps` muestran los 6 contenedores en ejecución, y las pruebas con `curl` devuelven 200.

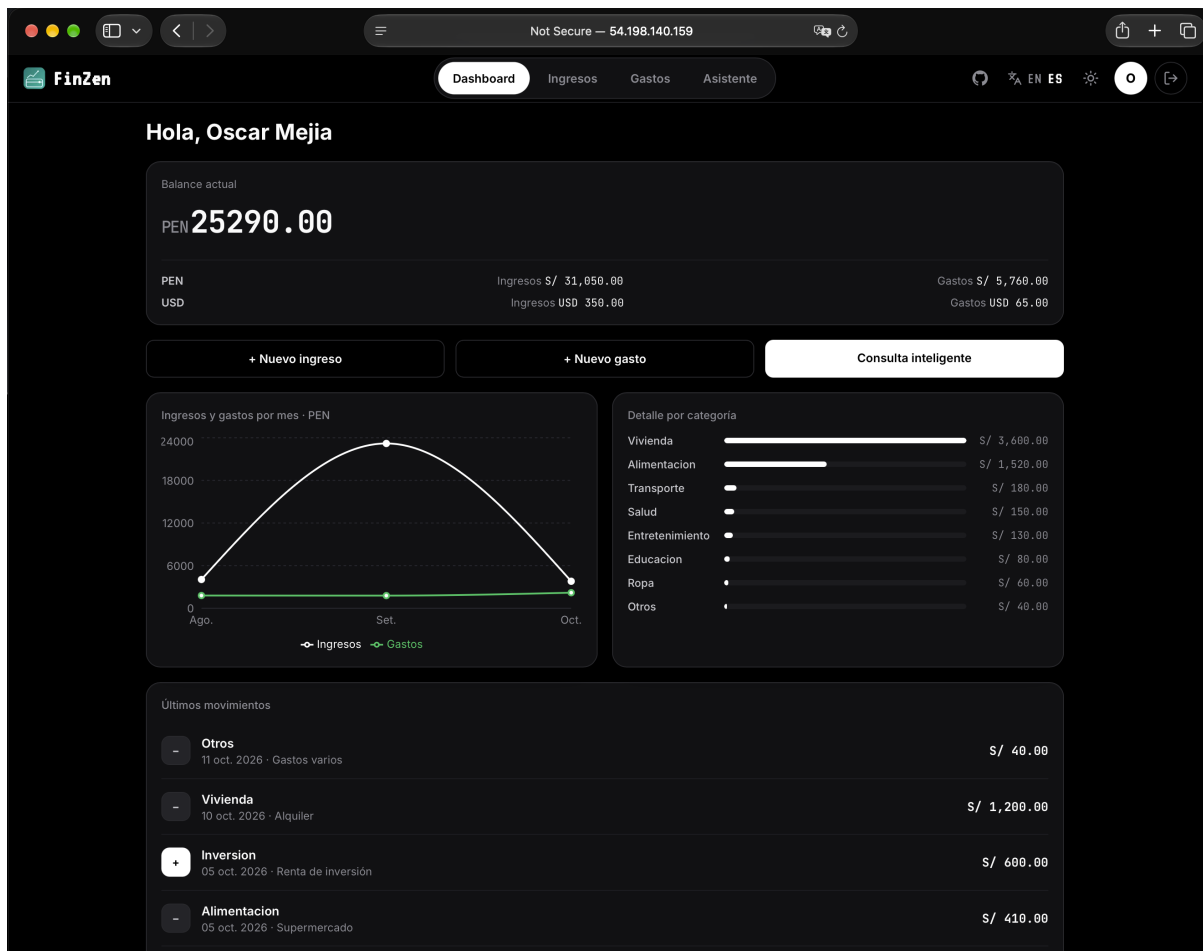


Figura 23: Navegador accediendo a la aplicación en <http://54.198.140.159>: panel de control del usuario autenticado.

19. Conclusiones

FinZen satisface el alcance del MVP propuesto: registro seguro de usuarios, captura de ingresos y gastos, listados y panel de control, y un asistente basado en RAG que responde sobre los datos del propio usuario. Se aplicaron prácticas de contenedores (imágenes livianas, usuario no privilegiado, *healthchecks* y volúmenes) y de seguridad (JWT, bcrypt y CORS restringido).

19.1. Trabajos Futuros

Aunque las 8 historias del MVP están liberadas, existen mejoras motivadas por escalabilidad o por la evolución del negocio:

- **Escalabilidad horizontal:** ante cientos o miles de usuarios concurrentes, pasar a **Application Load Balancer y Auto Scaling** sobre varias instancias y delegar la sesión en un almacén de sesiones distribuido.
- **Motor de datos:** migrar de SQLite a **PostgreSQL en Amazon RDS** cuando los volúmenes de movimientos superen el rendimiento de un archivo `.db` y sea necesario compartir el estado entre servidores.
- **Automatización del despliegue:** pasar de *user data* manual a un pipeline de CI/CD que construya las imágenes, las publique y actualice los servicios sin intervención manual.
- **Evolución del asistente:** incorporar *reranking* y `sqlite-vec` en el pipeline de RAG para mejorar la recuperación y la precisión de las respuestas.
- **Cobertura de APIs de edición:** el catálogo actual expone alta, lectura y eliminación; la edición viene cubierta solo en perfil y clave de OpenAI. Falta la edición granular de movimientos y categorías (PUT/PATCH), que se considera trabajo futuro y, por tanto, fuera del alcance del MVP.

Referencias

- Documentación de Docker y Docker Compose.
- Documentación de FastAPI y Pydantic.
- Documentación de AWS EC2 y calculadora de costos (<https://calculator.aws>).

- Documentación de OpenAI (modelos de embeddings y de chat).
- Documentación de Next.js (App Router).

Anexos

Los recursos complementarios del proyecto se encuentran en el repositorio de documentación [finzen-docs](#):

- **Anexo A** — Prompt del prototipo: [Link](#).
- **Anexo B** — Colección Postman y entornos: [Link](#).
- **Anexo C** — Catálogo completo de APIs: [Link](#).
- **Anexo D** — Repositorios de código: [finzen-app](#), [finzen-webui](#) y [finzen-docs](#).
- **Anexo E** — Aplicación desplegada: <http://54.198.140.159>.